elan1 Govern

ARCHITECTURE ATLAS · REVISED EDITION

Architecture and workflow figures — from system context to operations — for Elan1 Govern

COMPANION TO

the Elan1 Govern Technology Memorandum (revised 27 October 2026). Each figure is reproduced at full page with its caption and the chapter that explains it. Every figure states its status in the top-right corner and its sources along the foot; latency, cost and date figures are targets, not measurements.

REVISED EDITION · 27 OCTOBER 2026

The product formerly called elan1 is now **Elan1 Govern**, and it now exists as working code. The new figures (33 onward) show what is built and come first, on pages R-1 onward; original figures that are now outdated carry a "Revised" mark pointing to their replacement.

ENTITY	elan1 Private Limited
OWNER	M K Elango, Founder
STATUS	Revised edition — draft for review
DATE	27 October 2026 (first issued 23 September 2026)
SOURCE FILES	fig-01 … fig-32 (SVG) as issued; new figures 33 onward in this edition

ELAN1 · CONFIDENTIAL · FOR THE RECIPIENT NAMED ABOVE

Figures

SYSTEM CONTEXT AND ARCHITECTURE

Figure 1 — elan1 in context 3

Figure 2 — The perimeter and the seams 4

Figure 3 — Planes P0–P8 and who owns each 5

Figure 4 — Components, repositories and internal seams 6

THE KERNEL AND THE GOVERNED WRITE

Figure 5 — KernelPort v1 — six calls, five objects 7

Figure 6 — The Right Path — nine rungs in one order 8

Figure 7 — One governed action, as a sequence 9

Figure 8 — Two modes, two claims 10

DECLARATION AND APPROVAL

Figure 9 — The Declaration — seven blocks and their relations 11

Figure 10 — From clause to proof field — the five-link chain 12

Figure 11 — The approval path — obligation state machine 13

DATA, EVIDENCE AND CRYPTOGRAPHY

Figure 12 — Data architecture — stores, flows, tenancy and retention 14

Figure 13 — The Level 1 proof record — a per-tenant hash chain 15

Figure 14 — Seal Spec v6 — sixteen fields to a signed, anchored seal 16

Figure 15 — The evidence bundle and the offline verifier 17

Figure 16 — Key management and custody 18

Figure 17 — Two-authority timestamp anchoring 19

PACKS

Figure 18 — The pack factory: five gates, eleven blocks, eight tests 20

Figure 19 — ledger1 at runtime 21

Figure 20 — collect1: four obligation layers, fourteen gates 22

LEVEL 3 AND THE STANDARD

Figure 21 — Cross-organisation verification 23

Figure 22 — Selective disclosure: revealed versus committed 24

Figure 23 — Precedent and earned autonomy 25

Figure 24 — Action Receipt standard, certification and regulators 26

ASSURANCE, RUNTIME, SECURITY AND METERING

Figure 25 — The assurance ladder and the upgrade path 27

Figure 26 — Runtime and deployment topology 28

Figure 27 — Security, residency and DPDP controls by layer 29

Figure 28 — Metering pipeline: rung 9 to the invoice 30

STACK, BUILD AND OPERATIONS

Figure 29 — Technology stack by layer 31

Figure 30 — Build roadmap, October 2026 to March 2031 32

Figure 31 — CI pipeline and quality gates 33

Figure 32 — Operations: SLOs, failure behaviour and incidents 34

REVISION SUPPLEMENT · 27 OCTOBER 2026

What changed in this edition

The product is now Elan1 Govern

On 27 October 2026 the founder renamed the product: what these pages call **elan1** — the governance kernel — is now **Elan1 Govern**. In the original pages, 'elan1' meaning the product or kernel should be read as Elan1 Govern; 'elan1 Private Limited' remains the entity. Sister brands keep their names: rung1, stack1 and verna1, and the regulatory packs ledger1, invoice1 and collect1.

What is now built

Elan1 Govern is now working code: the repository Hasif-Elan1/elan1-govern, audited at commit 22388ad on 27 October 2026. Built is not shipped. There is no customer, production deployment, certification or third-party verification yet, and all data in the repository is seed or demo data.

Topic	23 September edition	27 October — in code	See
Scope	Governed writes into a system of record (ERP, Indian finance first)	Every agent tool call, reads and writes, in any industry	Fig 33
Enforcement	Mode A in-process and enforced; Mode B advisory over MCP or HTTP	**Governed run** (MCP proxy; the Enforced mode) enforces, because Elan1 Govern holds the credentials and runs the tool. **Check and record** (SDK or REST; the Attested advisory mode) is advisory. There is no in-process mode	Figs 33, 34
Decision	KernelPort v1 and nine rungs	One eight-step guard: signed rulebook; the strictest rule wins; no match means refuse; an error refuses. Known gap: allowed and approved MCP actions run before their receipt is written	Fig 34
Proof	L1 hash chain, not signed; signing and anchoring at L2	Every receipt SHA-256 + Ed25519; hourly signed checkpoints; offline `govern verify` (exit 0, 1, 2). No external anchoring, no KMS or HSM. L2 is not claimed; two of its ingredients — Ed25519 signing and an offline verifier — exist in code	Fig 35
Policy	Declaration (seven blocks) and the Studio	Rulebook, one per agent: Try an action, Test on past actions, signed versions, rule suggestions	Fig 36
Approval	Obligation state machine	Pending, approved, rejected, expired; signed single-use links; Slack and WhatsApp buttons only on rules marked for chat, and they skip the password check	Fig 37
Governance	Controls and regulations as specification	AI inventory, assessments, 10 frameworks, 13 controls, 7 industry packs — Draft, needs legal review	Fig 38
Operations	Stop switch, SLO targets	Six behaviour checks, automatic pause (off by default, high alerts only), emergency stop with an incident report; the stop does not cancel approvals already waiting	Fig 39
Screens	275 designed screens	13 areas; 130 built rows on 27 distinct page routes	Fig 40
Runtime	stack1 and Temporal in ap-south-1; hosting profiles	Two services, PostgreSQL and pg-boss, Docker; tested as a single instance (rate limits and tool cache per process); tenant isolation in application code; no residency control	Fig 42
Specified, not built	Seal Spec v6, RFC 3161 anchoring, KMS or HSM custody, assurance levels, Level 3, metering and billing, the regulatory pack factory and the regulatory packs, earned autonomy, residency and erasure		Fig 41

How to read the stamps

Original figures that the code has overtaken carry a stamp at the top right of their page. **SEE FIG 35** means the figure is outdated and replaced by the new figure named. **PARTLY · SEE FIG 41** means the idea still stands but the names or the mechanism differ. Unstamped figures (17–24 and 28) remain valid statements of plans and specifications; read 'elan1' in them as Elan1 Govern.

Status tags on the new figures

Each new figure states its status at the top right, in the same plain grey text as the originals. **BUILT — IN CODE** means present in the repository at commit 22388ad and confirmed by an independent read of the code; the demo was not run in the audit, and built does not mean shipped to a customer. **SPECIFIED** means a frozen specification with no code; dashed boxes mark these. Elan1 Govern is the current, partial Level 1 implementation (no meter; enforcement only on the MCP route); Level 2 is not claimed. Every compliance mapping is **Draft — needs legal review**.

Figures in this edition

No.	Figure	Page	Status	Superseded by
SYSTEM CONTEXT AND ARCHITECTURE				
1	elan1 in context	3	OUTDATED	Fig 33
2	The perimeter and the seams	4	PARTLY OUTDATED	Figs 33, 42
3	Planes P0–P8 and who owns each	5	PARTLY OUTDATED	Fig 42
4	Components, repositories and internal seams	6	OUTDATED	Fig 42
THE KERNEL AND THE GOVERNED WRITE				
5	KernelPort v1 — six calls, five objects	7	PARTLY OUTDATED	Figs 34, 41
6	The Right Path — nine rungs in one order	8	PARTLY OUTDATED	Fig 34
7	One governed action, as a sequence	9	PARTLY OUTDATED	Figs 34, 37
8	Two modes, two claims	10	PARTLY OUTDATED	Fig 34
DECLARATION AND APPROVAL				
9	The Declaration — seven blocks and their relations	11	PARTLY OUTDATED	Fig 36
10	From clause to proof field — the five-link chain	12	PARTLY OUTDATED	Fig 38
11	The approval path — obligation state machine	13	OUTDATED	Fig 37
DATA, EVIDENCE AND CRYPTOGRAPHY				
12	Data architecture — stores, flows, tenancy and retention	14	PARTLY OUTDATED	Fig 42
13	The Level 1 proof record — a per-tenant hash chain	15	OUTDATED	Fig 35
14	Seal Spec v6 — sixteen fields to a signed, anchored seal	16	PARTLY OUTDATED	Fig 35
15	The evidence bundle and the offline verifier	17	PARTLY OUTDATED	Fig 35
16	Key management and custody	18	PARTLY OUTDATED	Fig 35
17	Two-authority timestamp anchoring	19	STILL VALID	—
PACKS				
18	The pack factory: five gates, eleven blocks, eight tests	20	STILL VALID	—
19	ledger1 at runtime	21	STILL VALID	—
20	collect1: four obligation layers, fourteen gates	22	STILL VALID	—
LEVEL 3 AND THE STANDARD				
21	Cross-organisation verification	23	STILL VALID	—
22	Selective disclosure: revealed versus committed	24	STILL VALID	—
23	Precedent and earned autonomy	25	STILL VALID	—
24	Action Receipt standard, certification and regulators	26	STILL VALID	—
ASSURANCE, RUNTIME, SECURITY AND METERING				
25	The assurance ladder and the upgrade path	27	PARTLY OUTDATED	Fig 41
26	Runtime and deployment topology	28	OUTDATED	Fig 42
27	Security, residency and DPDP controls by layer	29	PARTLY OUTDATED	Fig 42
28	Metering pipeline: rung 9 to the invoice	30	STILL VALID	—
STACK, BUILD AND OPERATIONS				
29	Technology stack by layer	31	OUTDATED	Fig 42
30	Build roadmap, October 2026 to March 2031	32	PARTLY OUTDATED	Fig 41
31	CI pipeline and quality gates	33	PARTLY OUTDATED	Fig 42
32	Operations: SLOs, failure behaviour and incidents	34	PARTLY OUTDATED	Fig 39
REVISION SUPPLEMENT — ELAN1 GOVERN AS BUILT (27 OCTOBER 2026)				
33	Elan1 Govern in context	R-3	Built — in code	
34	The decision path for one tool call	R-4	Built — in code	
35	Receipts, checkpoints and offline verify	R-5	Built — in code · anchoring, KMS specified	
36	The rulebook lifecycle and the suggestions loop	R-6	Built — in code	
37	The approval path, links and chat	R-7	Built — in code	
38	The governance layer	R-8	Built — in code · draft, needs legal review	
39	Alerts, automatic pause and the emergency stop	R-9	Built — in code	
40	One information architecture: thirteen areas	R-10	Mapping — screen map of 27 Oct 2026	
41	Spec-to-code crosswalk	R-11	Built vs specified	
42	Deployment and package topology	R-12	Built — in code · hosting, residency specified	

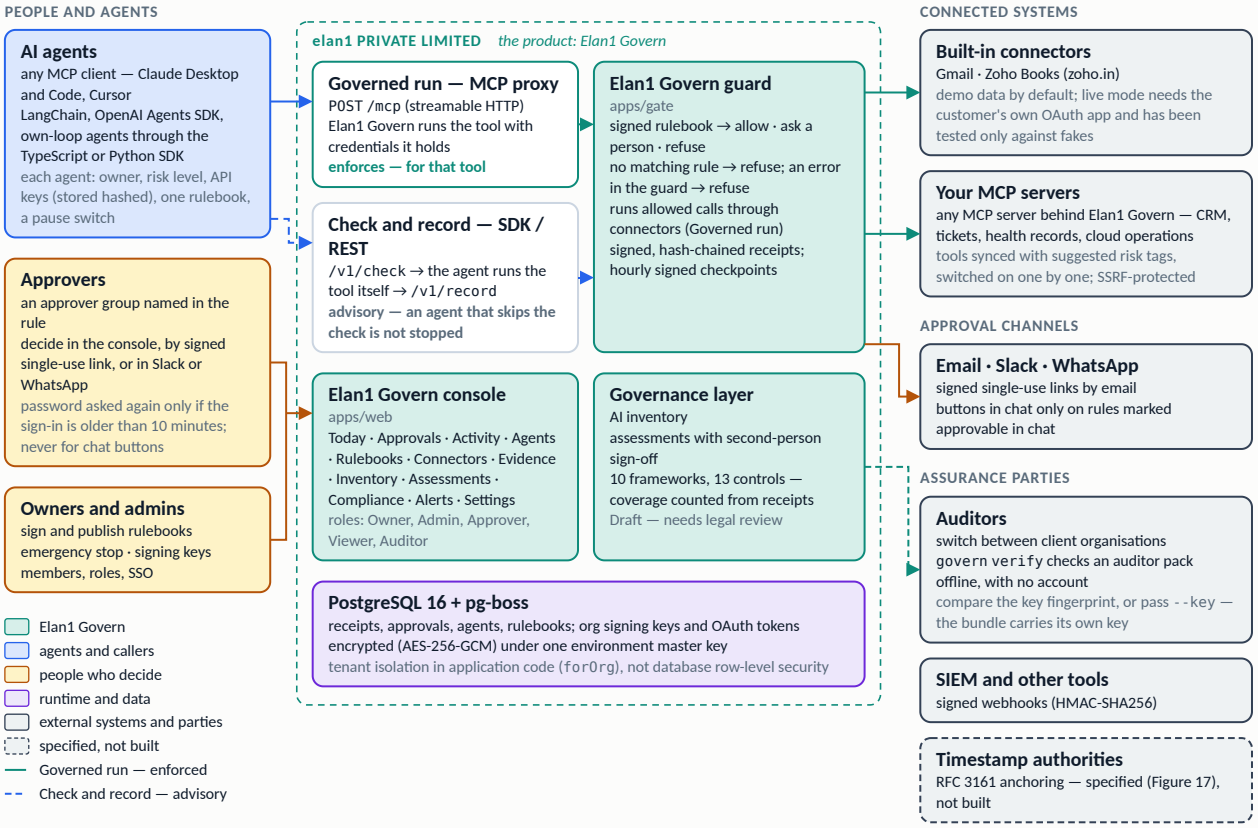
Pages 1–34 are the original edition of 23 September 2026, unchanged except for the stamps. Pages R-1 to R-12 are this supplement.

FIGURE 33 OF 42

Figure 33 — Elan1 Govern in context

BUILT — IN CODE · ANCHORING — SPECIFIED

Who calls it, who decides, what it governs and who checks it — every tool call, from any agent, in any industry



Only a Governed run enforces: Elan1 Govern holds the downstream credentials and runs the tool. Check and record is advisory — the agent holds its own credentials, and nothing in the code stops an agent that skips the check.

Source: docs/ARCHITECTURE.md §System overview, §Packages; apps/gate/src/guard.ts (GuardMode), mcp.ts, routes/agent-api.ts; code audit of commit 22388ad (27 Oct 2026) · Elan1 Govern Technology Memorandum (revised) · Oct 2026

Figure 33 — Elan1 Govern in context: agents call their tools through Elan1 Govern either as a Governed run over MCP, where Elan1 Govern holds the credentials and runs the tool, or through Check and record over the SDK or REST, where the agent acts on the answer itself; the guard decides allow, ask a person or refuse from a signed rulebook and signs a receipt; approvers decide by link, Slack or WhatsApp; auditors verify offline.

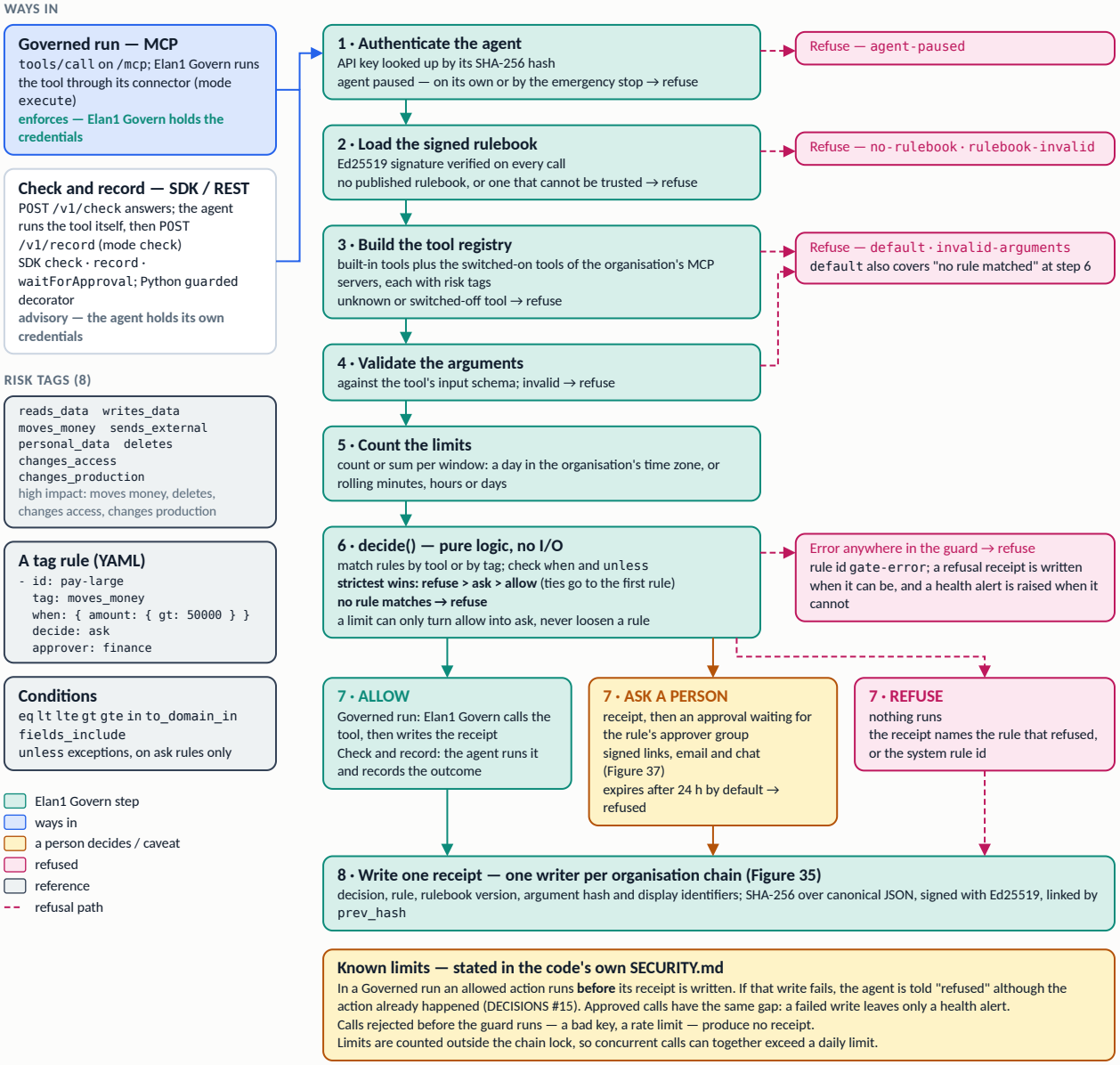
Explained in: Technology Memorandum, Revision supplement §R1 Architecture, packages, stack and deployment · Source file: fig-33-govern-in-context.svg

FIGURE 34 OF 42

Figure 34 — The decision path for one tool call

BUILT — IN CODE

Every call takes the same steps in one order; the strictest rule wins, no matching rule means refuse, and an error refuses



One path for every call, read or write. A tag rule governs tools Elan1 Govern has never seen, which is what lets one rulebook work across industries.

Source: apps/gate/src/guard.ts; packages/core/src/engine.ts, rulebook.ts, tools.ts; docs/ARCHITECTURE.md §Invariants 1–3, §The decision pipeline; SECURITY.md Known limits; code audit of commit 22388ad (27 Oct 2026) · Elan1 Govern Technology Memorandum (revised) · Oct 2026

Figure 34 — The decision path for one tool call: the agent's key, the signed rulebook, the tool registry, the argument check and the limits come before a pure decision engine in which the strictest matching rule wins and no match means refuse; every earlier step can only refuse, an error refuses, and every decision the guard makes ends in a signed receipt. Only a Governed run over MCP enforces the decision.

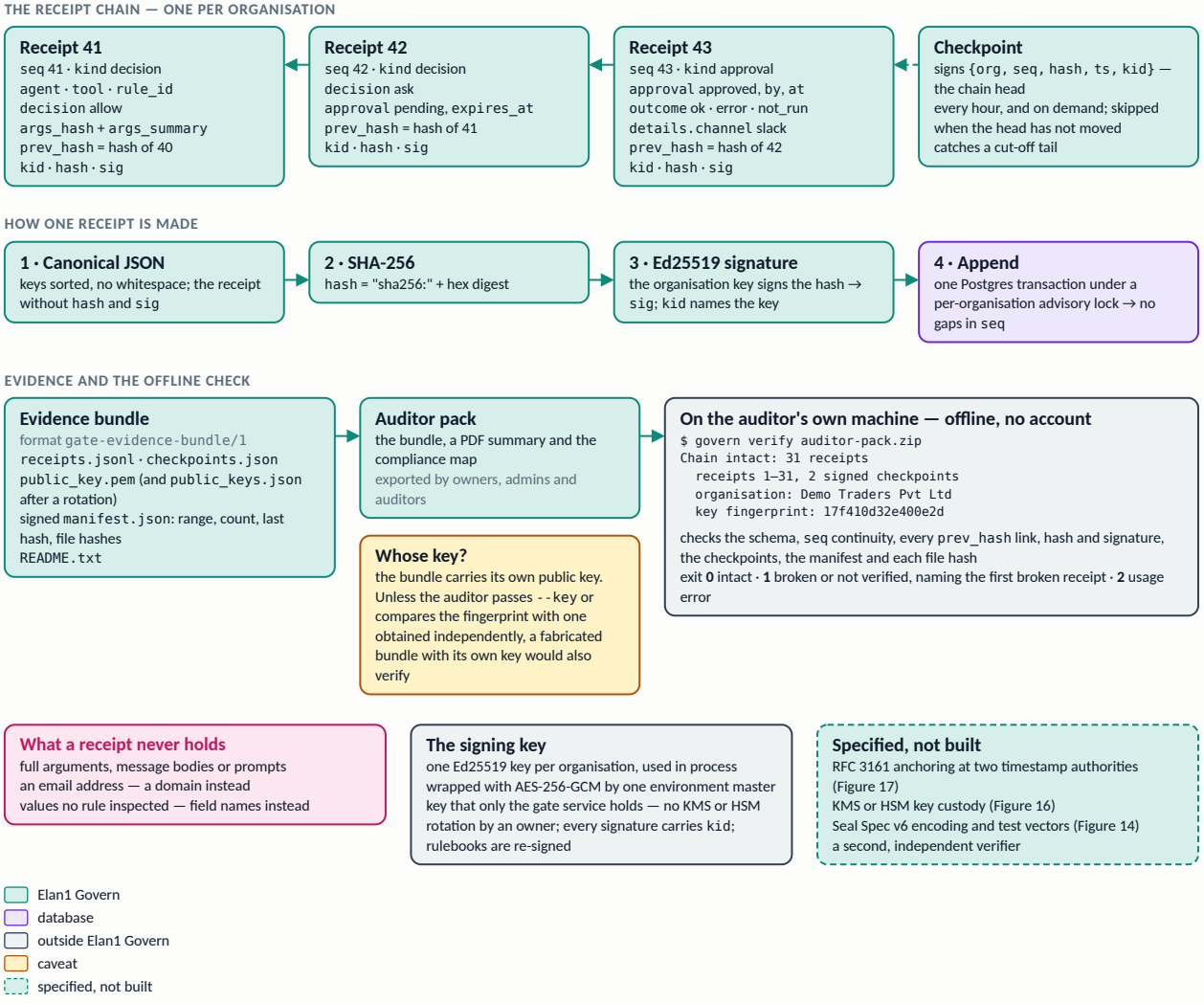
Explained in: Technology Memorandum, Revision supplement §R2 Decision path and data model · Source file: fig-34-decision-path.svg

FIGURE 35 OF 42

Figure 35 — Receipts, checkpoints and offline verify

BUILT — IN CODE · ANCHORING, KMS — SPECIFIED

Every decision is a signed receipt linked to the one before; signed checkpoints fix the chain head; one command checks it all offline



Receipts are signed today. What Level 2 adds on top — external timestamps, managed key custody and the Seal v6 encoding — is still specified, not built.

Source: packages/core/src/receipt.ts, canonical.ts, crypto.ts, bundle.ts; packages/db/src/chain.ts; packages/cli/src/cli.ts; apps/gate/src/evidence.ts, jobs.ts, keys.ts; code audit of commit 22388ad (27 Oct 2026) · Elan1 Govern Technology Memorandum (revised) · Oct 2026

Figure 35 — The receipt chain: each decision is canonical JSON hashed with SHA-256, signed with the organisation's Ed25519 key and linked to its predecessor under a single writer; signed checkpoints fix the chain head every hour; the evidence bundle and auditor pack carry receipts, checkpoints, the public key and a signed manifest, which govern verify checks offline, naming the first broken receipt.

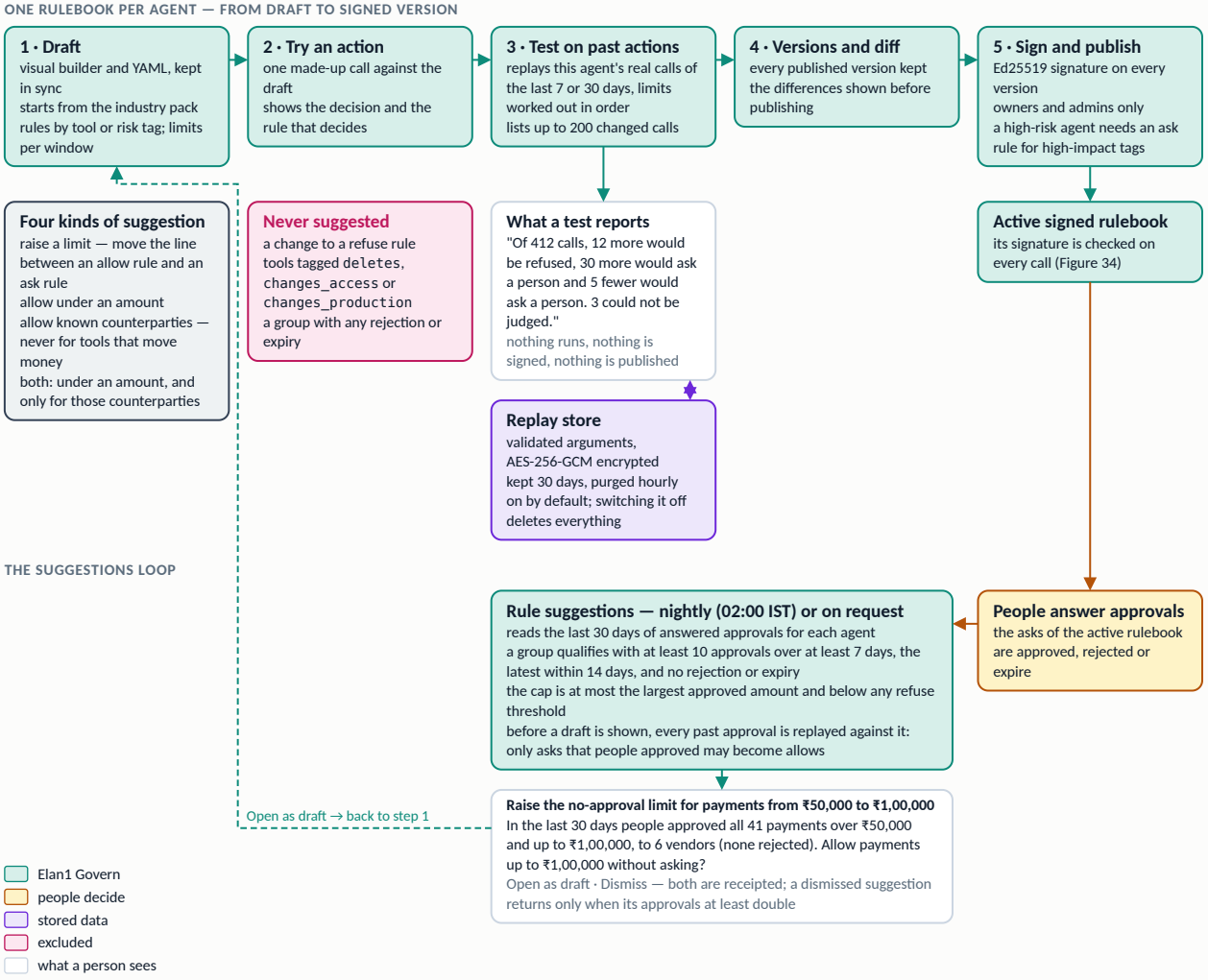
Explained in: Technology Memorandum, Revision supplement §R3 Receipts, checkpoints, bundles, verify and keys · Source file: fig-35-receipts-checkpoints-verify.svg

FIGURE 36 OF 42

Figure 36 — The rulebook lifecycle and the suggestions loop

BUILT — IN CODE

Draft, try, test on past actions, compare, sign and publish — and a nightly miner that drafts the rules people keep approving



Neither a test nor a suggestion changes anything: only a person who may publish signs a new rulebook version. Suggestions are built from values already stored with approvals — amounts, vendor or domain identifiers, the tool — never from message bodies.

Source: packages/core/src/rulebook.ts, rulebook-sign.ts, replay.ts, suggest.ts, diff.ts; apps/gate/src/replay.ts, suggestions.ts, jobs.ts; README §Test a rulebook on past actions, §Rule suggestions; code audit of commit 22388ad (27 Oct 2026) · Elan1 Govern Technology Memorandum (revised) · Oct 2026

Figure 36 — The rulebook lifecycle: a draft built visually or in YAML is tried on one made-up call, replayed against the agent's real calls of the last 7 or 30 days, compared with the published version and then signed and published; each night a miner turns groups of actions that people always approved into a suggested draft, which a person opens or dismisses.

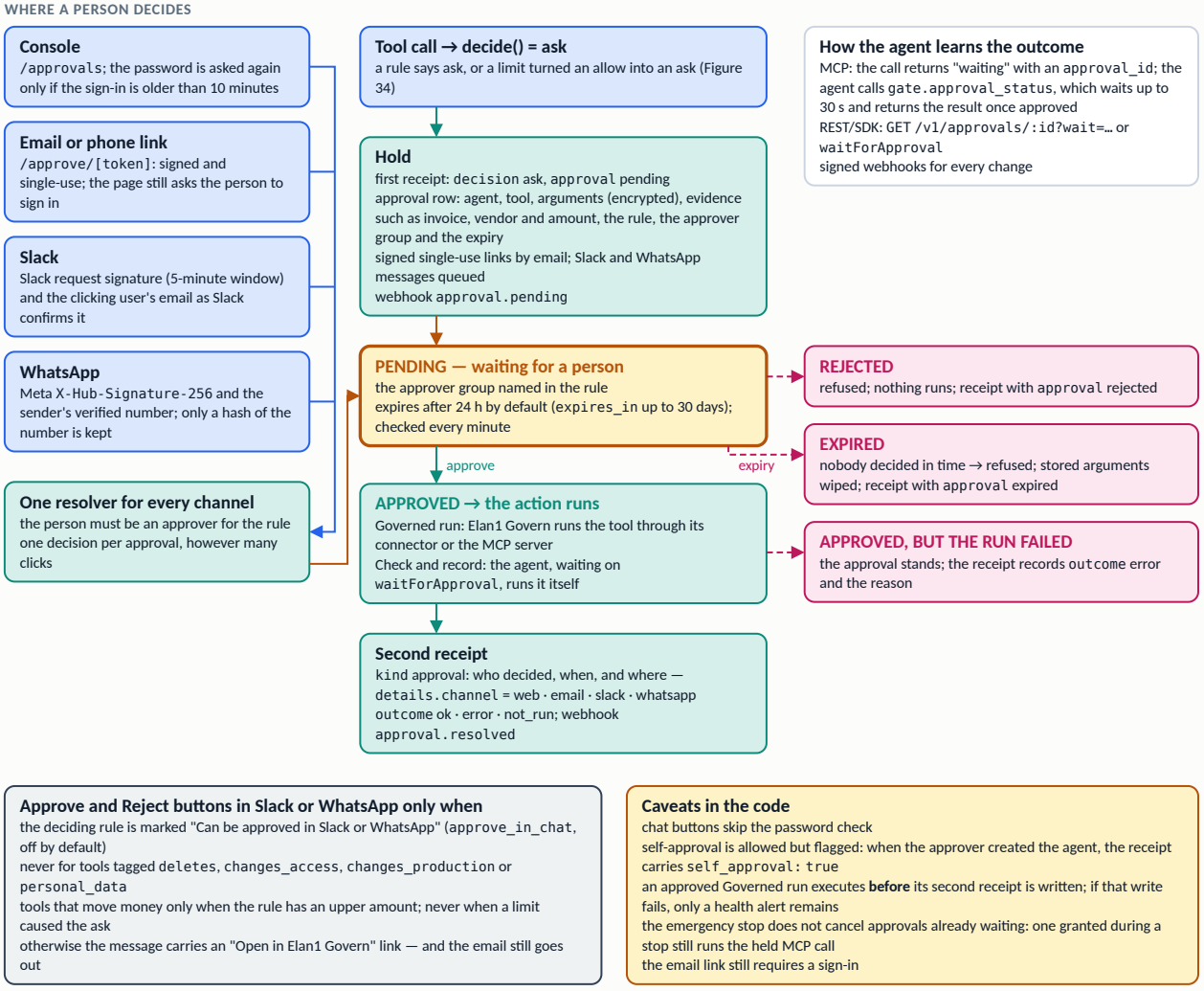
Explained in: Technology Memorandum, Revision supplement §R4 Rulebooks, testing, suggestions and industry packs · Source file: fig-36-rulebook-lifecycle.svg

FIGURE 37 OF 42

Figure 37 — The approval path, links and chat

BUILT — IN CODE

An action that needs a person waits; there is one decision per approval wherever it is made, and if nobody acts the action is refused



Slack and WhatsApp are a doorbell, not a replacement: the rule, the decision and the receipt stay in Elan1 Govern, and the same resolver decides whatever channel the click came from.

Source: apps/gate/src/approvals.ts, mcp.ts, chat/, routes/chat.ts, routes/agent-api.ts; packages/core/src/chat.ts; README §Approvals in Slack and WhatsApp; code audit of commit 22388ad (27 Oct 2026) · Elan1 Govern Technology Memorandum (revised) · Oct 2026

Figure 37 — The approval path as built: a held action waits for the approver group named in the rule; a decision in the console, by a signed single-use link, in Slack or in WhatsApp goes through one resolver, and approval runs the action and writes a second receipt recording where it was decided; rejection and expiry both refuse; chat buttons appear only on rules marked approvable in chat.

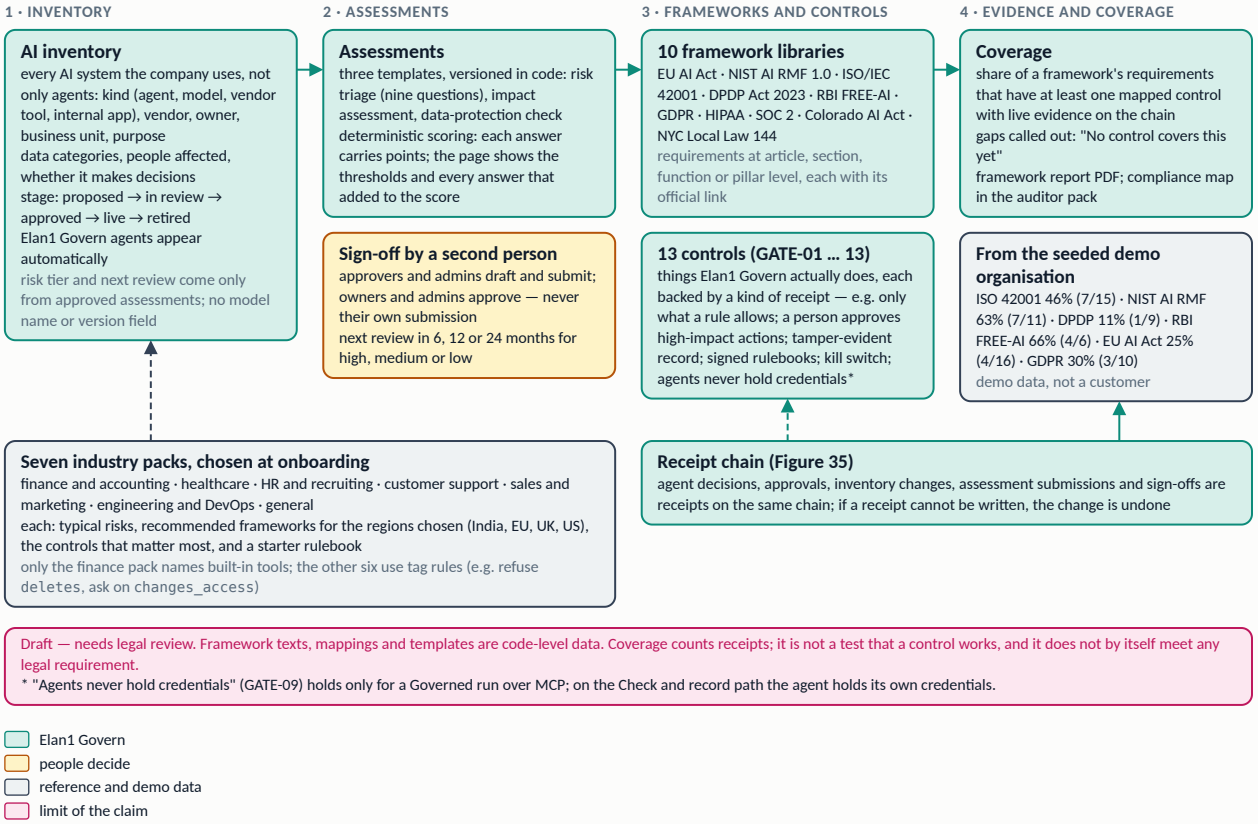
Explained in: Technology Memorandum, Revision supplement §R5 Approvals and Slack/WhatsApp · Source file: fig-37-approval-state-machine.svg

FIGURE 38 OF 42

Figure 38 — The governance layer

BUILT — IN CODE · DRAFT — NEEDS LEGAL REVIEW

Inventory → assessments → frameworks → evidence: coverage is counted from receipts, and everything is labelled Draft — needs legal review



Evidence is computed from receipts, not written by hand: the same chain that records each agent decision feeds the coverage of every framework turned on.

Source: packages/core/src/inventory.ts, assessments.ts, frameworks/frameworks.ts, controls.ts, packs.ts; README §AI inventory and assessments, §Frameworks and industry packs; docs/screenshots/17-compliance.png; code audit of commit 22388ad (27 Oct 2026) · Elan1 Govern Technology Memorandum (revised) · Oct 2026

Figure 38 — The governance layer: every AI system is registered with its owner and stage; structured assessments are scored deterministically and signed off by a second person; ten framework libraries map to thirteen controls whose evidence is counted from receipts on the chain, with gaps called out and every output labelled Draft — needs legal review.

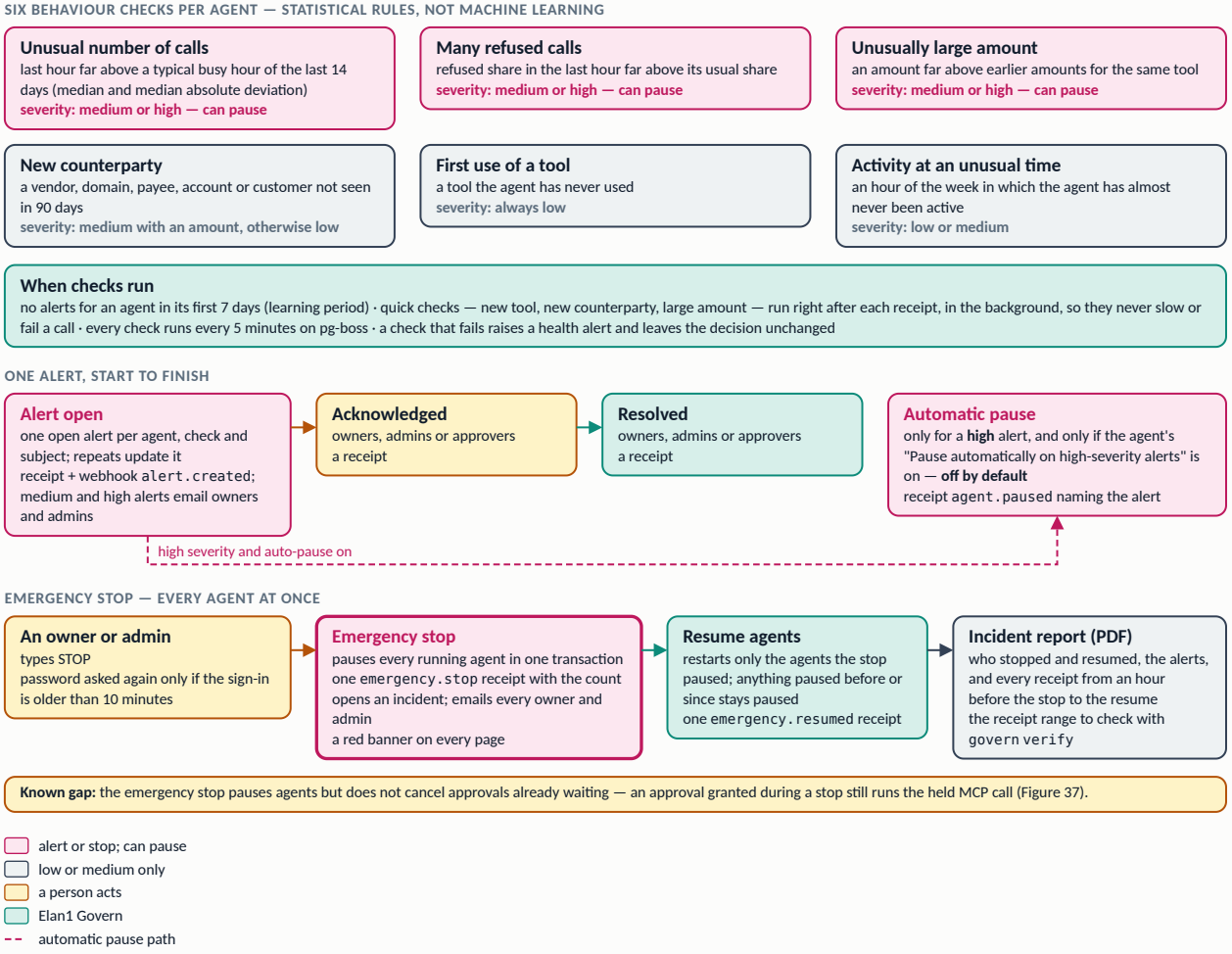
Explained in: Technology Memorandum, Revision supplement §R7 Governance layer · Source file: fig-38-governance-layer.svg

FIGURE 39 OF 42

Figure 39 — Alerts, automatic pause and the emergency stop

BUILT — IN CODE

Elan1 Govern learns what is normal for each agent; a high alert can pause one agent if its owner switched that on; STOP pauses them all



A paused agent is refused at the first step of every call (Figure 34). Only 3 of the 6 checks can raise a high alert, so only those three can pause an agent automatically, and only where its owner switched automatic pause on.

Source: packages/core/src/behaviour.ts (DETECTORS, BEHAVIOUR_DEFAULTS); apps/gate/src/behaviour.ts, incidents.ts, jobs.ts; apps/web/src/app/api/emergency-stop/route.ts; README §Unusual-behaviour alerts and the emergency stop; code audit of commit 22388ad (27 Oct 2026) · Elan1 Govern Technology Memorandum (revised) · Oct 2026

Figure 39 — Alerts and the emergency stop: six per-agent behaviour checks run after each receipt and every five minutes; each alert is receipted, acknowledged and resolved; a high alert pauses one agent only where automatic pause is switched on; the emergency stop pauses every agent in one transaction, writes one receipt, opens an incident and produces a report whose receipt range can be verified offline; it does not cancel approvals already waiting.

Explained in: Technology Memorandum, Revision supplement §R8 Alerts, auto-pause and emergency stop · Source file: fig-39-alerts-pause-stop.svg

FIGURE 40 OF 42

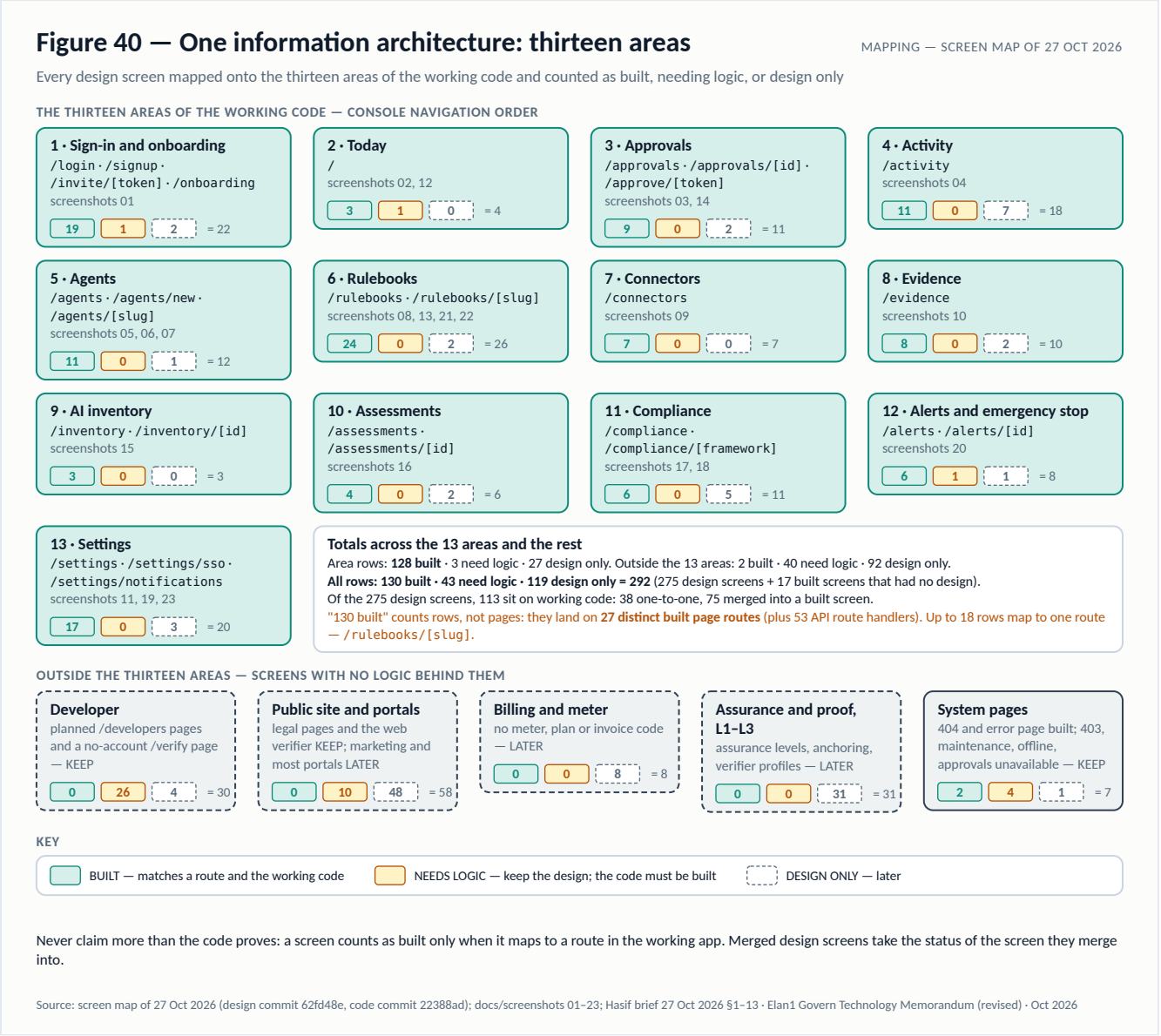


Figure 40 — One information architecture: the 275 design screens and 17 built screens without a design, mapped onto the thirteen areas of the working code and counted as built, needing logic or design only; the 130 built rows land on 27 distinct page routes, and the screens outside the thirteen areas are tagged keep or later.

Explained in: Technology Memorandum, Revision supplement §R14 Unified information architecture · Source file: fig-40-information-architecture.svg

FIGURE 41 OF 42

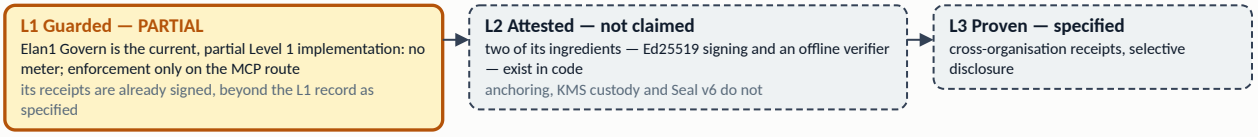
Figure 41 — Spec-to-code crosswalk

BUILT vs SPECIFIED — AS AT 27 OCT 2026

Each concept of the September specification beside its Elan1 Govern name and its place in the code (paths under the Elan1 Govern repository)

SPECIFIED (SEPT 2026 EDITION)	ELAN1 GOVERN — BUILT NAME	WHERE IN THE CODE	STATUS
Declaration, seven blocks (Fig 9)	Rulebook — one per agent, YAML, signed	`core/src/rulebook.ts` · `rulebook-sign.ts`	BUILT DIFFERENTLY — the seven-block ontology is not in the code
Write type	Tool + 8 risk tags	`core/src/tools.ts`	BUILT DIFFERENTLY — every call, reads included
Studio: simulate · certify	Try an action · Test on past actions · Sign and publish	`core/src/replay.ts` · `gate/src/replay.ts`	BUILT
Proof record — L1 hash chain (Fig 13)	Receipt: SHA-256 + Ed25519 signature	`core/src/receipt.ts` · `db/src/chain.ts`	BUILT — signed today; the L1 spec was unsigned
Chain standing	Receipt chain + hourly signed checkpoints	`db/src/chain.ts` · `gate/src/jobs.ts`	BUILT
Evidence bundle + offline verifier (Fig 15)	Auditor pack + `govern verify`	`core/src/bundle.ts` · `packages/cli`	PARTIAL — exit codes 0/1/2, not six; no verifier profiles
Approval obligation (Fig 11)	Approval: pending · approved · rejected · expired	`gate/src/approvals.ts`	BUILT DIFFERENTLY
WhatsApp approval	Slack and WhatsApp approvals	`gate/src/chat/` · `core/src/chat.ts`	BUILT
Stop switch — rung 0b	Emergency stop + per-agent pause	`web api/emergency-stop` · `gate/src/incidents.ts`	BUILT
Agent registry · Tools catalogue	Agents · Tool registry	`web (app)/agents` · `core/src/tools.ts`	BUILT
Connected systems	Connectors: Gmail, Zoho Books, Your MCP servers	`packages/connectors` · `gate/src/upstream.ts`	BUILT — live Gmail and Zoho never run against real accounts
Controls & regulations (Fig 10)	Compliance: 10 frameworks, 13 controls	`core/src/frameworks/`	BUILT — Draft — needs legal review
Regulatory packs (Fig 18)	Industry packs (7) — a different thing	`core/src/frameworks/packs.ts`	BUILT DIFFERENTLY — regulatory packs ledger1, invoice1, collect1 remain specified
Tenants	Organisations	`db/src/tenant.ts (forOrg)`	BUILT — app-level isolation; no Postgres RLS
MCP server, 34 tools in three scopes (spec)	MCP server `/mcp`: built-in, upstream and status tools	`gate/src/mcp.ts`	PARTIAL — not the specified 34-tool set
KernelPort v1 — six calls (Fig 5)	MCP `/mcp` + REST `/v1/*` (OpenAPI 3.1)	`gate/src/mcp.ts` · `routes/agent-api.ts`	BUILT DIFFERENTLY — no single port abstraction
Nine rungs (Fig 6)	Eight-step guard (Figure 34)	`gate/src/guard.ts`	PARTIAL — no write door, autonomy or metering rung
Mode A enforced / Mode B advisory (Fig 8)	Governed run (MCP proxy) enforces · Check and record (SDK/REST) is advisory	`gate/src/guard.ts (GuardMode)`	BUILT DIFFERENTLY — no in-process enforced mode
Single writer holds the target credential	Govern holds connector credentials in a Governed run	`gate/src/connectors.ts` · `upstream.ts`	PARTIAL — nothing stops an agent holding its own
Durable execution — Temporal	pg-boss jobs in Postgres	`gate/src/jobs.ts`	BUILT DIFFERENTLY
Seal Spec v6, 64 vectors, two implementations (Fig 14)	—		SPECIFIED — frozen spec; no code
RFC 3161 two-authority anchoring (Fig 17)	— (checkpoints are self-signed)		SPECIFIED
KMS / HSM key custody (Fig 16)	One env master key wraps org keys	`gate/src/keys.ts (KeyProvider)`	SPECIFIED — a code seam only
Earned autonomy (Fig 23)	nearest: rule suggestions, published by a person	`core/src/suggest.ts`	SPECIFIED
Metering and billing (Fig 28) · assurance levels and L3 (Figs 21–25) · residency, erasure, idempotency keys	—		SPECIFIED

WHERE THE BUILT CODE SITS AGAINST THE SEPTEMBER ASSURANCE LADDER



Where the code and the September specification differ, the code is what exists and the specification is what is planned. SPECIFIED means a frozen specification with no code. Status follows the code audit of 27 October 2026 (commit 22388ad).

Source: code audit of commit 22388ad (27 Oct 2026); Hasif brief 27 Oct 2026 (name table); docs/ARCHITECTURE.md; Atlas Figures 5–28 (Sept 2026) · Elan1 Govern Technology Memorandum (revised) · Oct 2026

Figure 41 — Spec-to-code crosswalk: each concept of the September specification beside its Elan1 Govern name and its place in the code, marked built, built differently, partial or specified (frozen specification with no code), and where the built product sits on the September assurance ladder: the current, partial Level 1 implementation, with Level 2 not claimed.

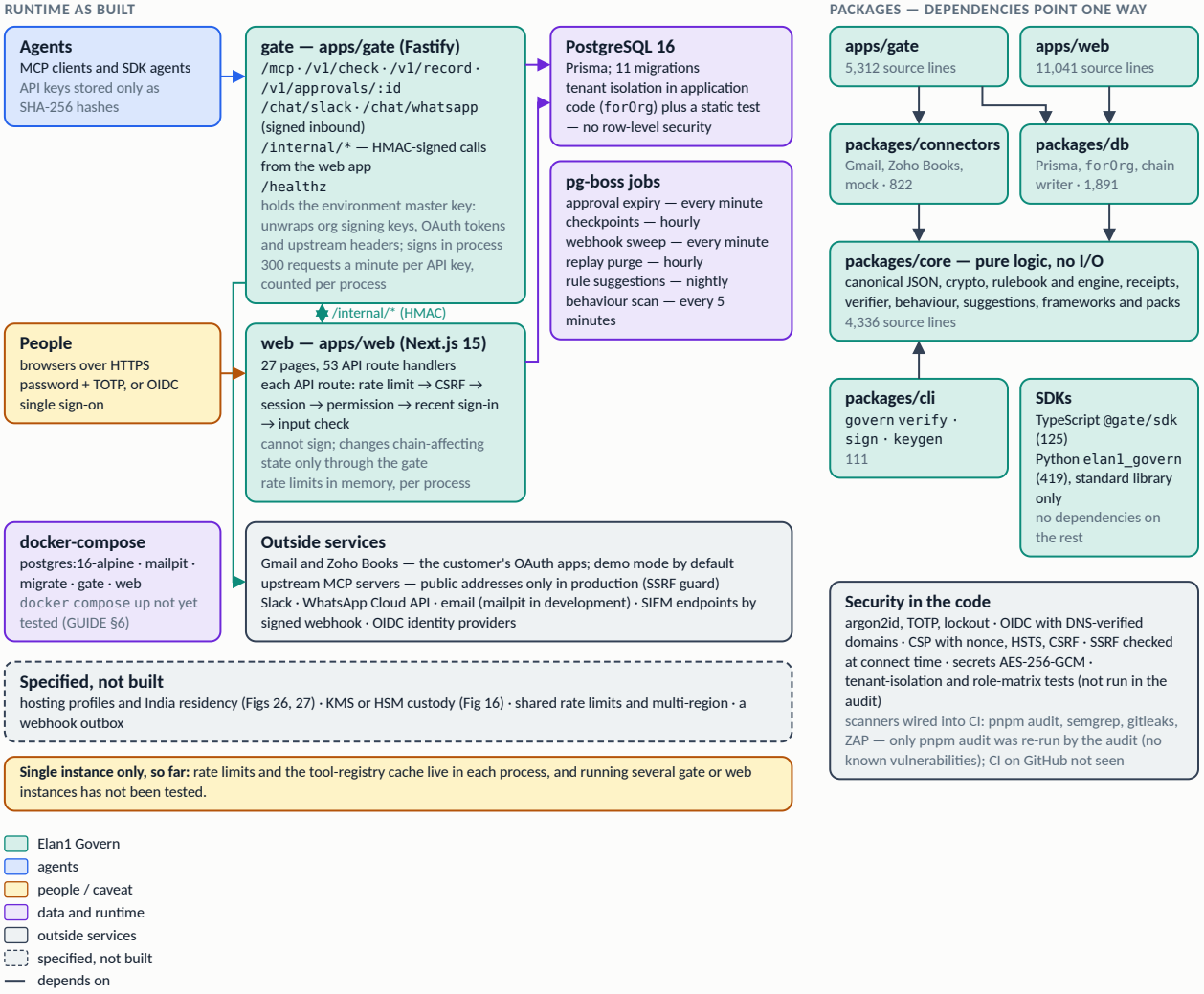
Explained in: Technology Memorandum, Revision supplement §R12 Spec-to-code crosswalk · Source file: fig-41-spec-to-code-crosswalk.svg

FIGURE 42 OF 42

Figure 42 — Deployment and package topology

BUILT — IN CODE · HOSTING, RESIDENCY — SPECIFIED

Two services and one PostgreSQL that also carries the job queues; tested as a single instance; package dependencies point one way



Configuration is environment only and checked at start-up; in production, missing secrets or the sample values stop the process. There is no production deployment yet: all data in the repository is seed or demo data.

Source: docs/ARCHITECTURE.md §System overview, §Packages, §Operations; docker-compose.yml; Dockerfile; apps/gate/src/app.ts, jobs.ts, keys.ts; apps/web/src/server/api.ts; SECURITY.md; code audit of commit 22388ad (27 Oct 2026) · Elan1 Govern Technology Memorandum (revised) · Oct 2026

Figure 42 — Deployment and packages as built: the gate service and the web console share one PostgreSQL that also carries the background job queues; only the gate holds the master key and talks to outside services; the console changes chain-affecting state only through signed internal calls; rate limits and the tool-registry cache are per process, so only a single instance is tested; package dependencies run one way, from the apps down to the pure core.

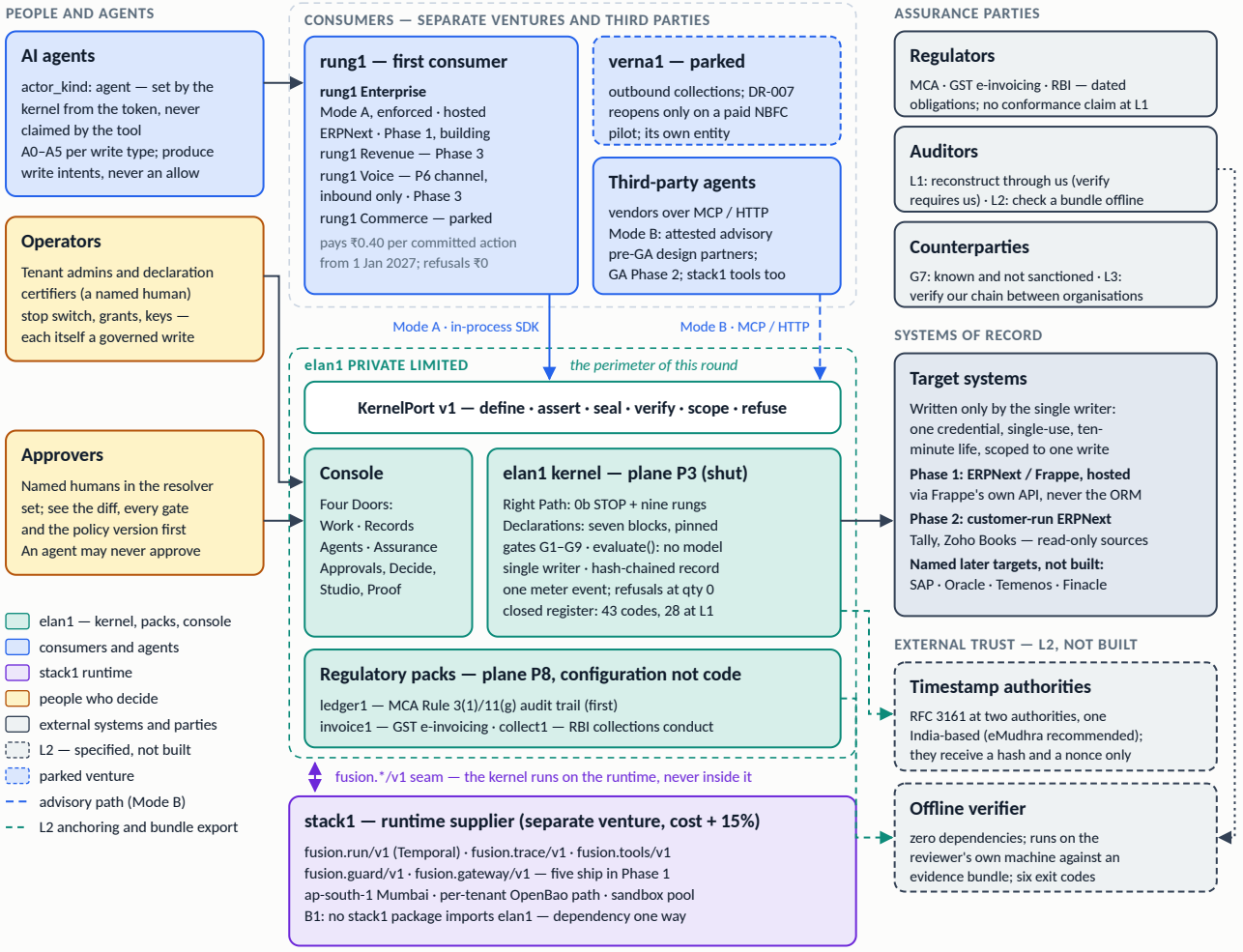
Explained in: Technology Memorandum, Revision supplement §R1 Architecture, packages, stack and deployment · Source file: fig-42-deployment-packages.svg

FIGURE 1 OF 32

Figure 1 — elan1 in context

L1 — BEING BUILT · L2/L3 — NOT BUILT

Who acts, who consumes the kernel, what it runs on, and the external systems it writes to or is checked by



A model may produce a write intent; it may never produce an allow. In Mode A only the kernel's single writer holds a credential to the books.

Source: E01 §02–05, E02 §01, E04, E05 §02, E06, E17 §03, O03 §01–02, O01 §02, S03 §02, R10, IM §1.3–1.4, §2.1–2.3, §6.1, App. B · elan1 Technology Memorandum · Sept 2026

Figure 1 — elan1 in context: the actors who ask, approve and audit; the consumers that build against KernelPort; stack1 beneath as the runtime; the systems of record written through the single writer; and the trust services used from Level 2.

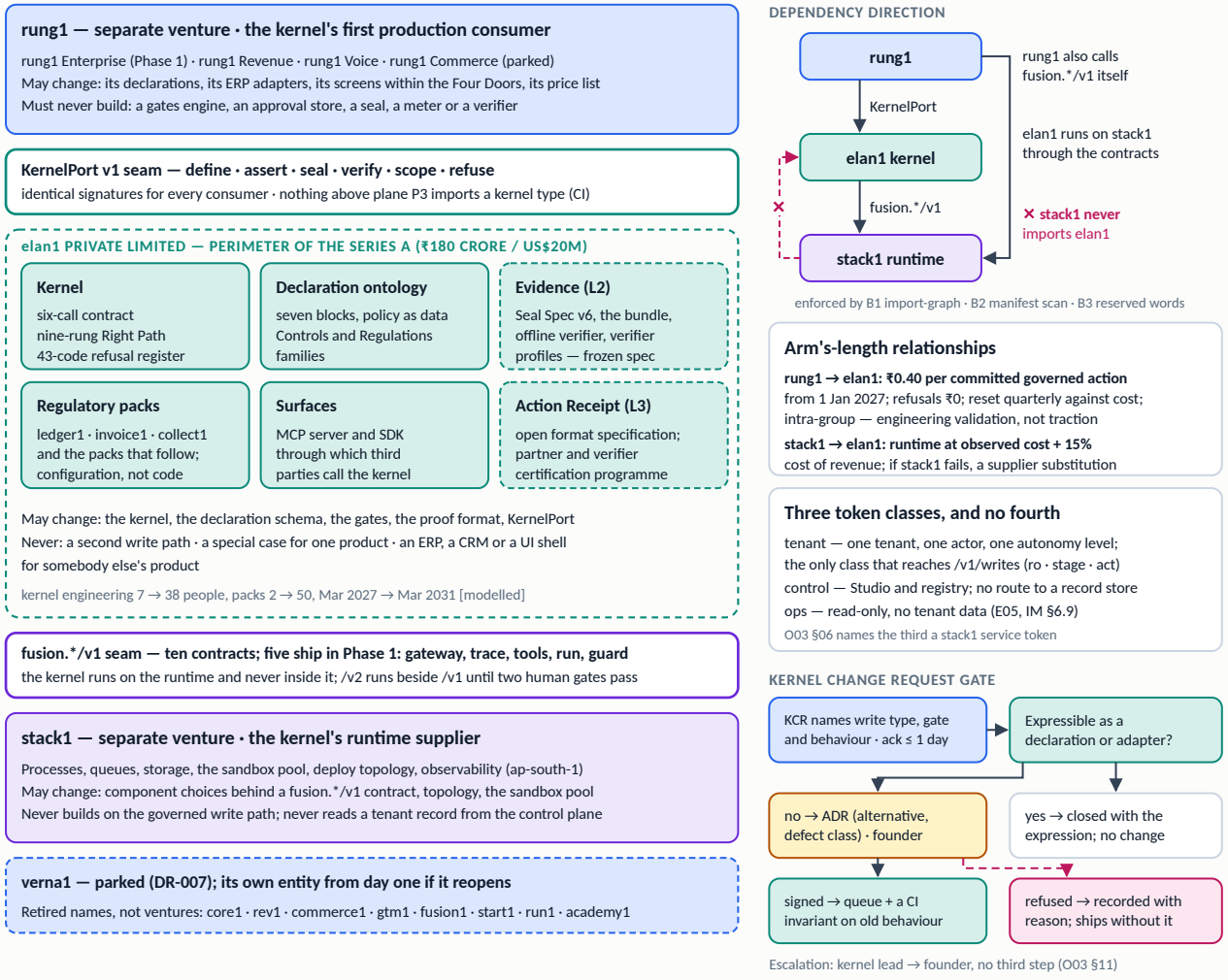
Explained in: 2. System context, design principles and the perimeter (memorandum page 10) · Source file: fig-01-system-context.svg

FIGURE 2 OF 32

Figure 2 — The perimeter and the seams

L1 — BEING BUILT

What elan1 Private Limited holds, the two separate ventures on either side, the two seams and the gate on kernel change



One kernel, reached through six calls, running on a runtime it never lives inside. A product team that ships its own gate has built a second kernel.

Source: IM §2.1–2.3, §6.1–6.2, §6.9, App. B; O03 §02, §06, §09–12; O11 §02; E02 §01, §04; E05 §05; S01 §02; S03 §02–03 · elan1 Technology Memorandum · Sept 2026

Figure 2 — The perimeter and the Three-Brand seam: elan1 Private Limited (kernel and packs) between rung1 above and stack1 below, with the one-way dependency direction, the KernelPort and fusion.*/v1 seams, and the three token classes.

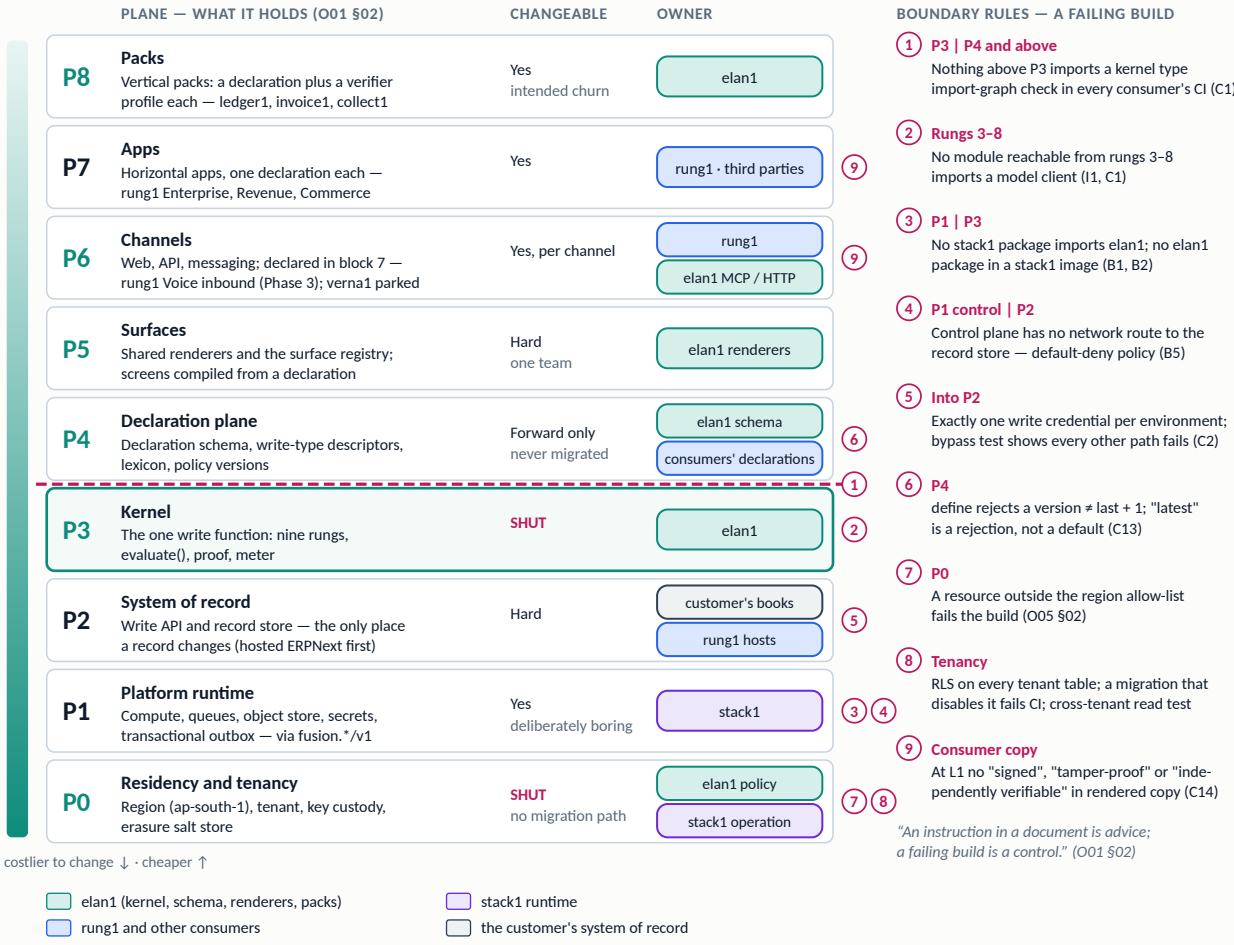
Explained in: 2. System context, design principles and the perimeter (memorandum page 15) · Source file: fig-02-perimeter-seam.svg

FIGURE 3 OF 32

Figure 3 — Planes P0–P8 and who owns each

L1 — BEING BUILT

Nine planes read bottom-up as a cost gradient, the brand that owns each, and the boundary rules a failing build enforces



Only two planes are shut: P0, because a residency or key-custody choice has no migration path, and P3, because everything depends on it.

Source: O01 §02, DR-011; O03 §10, §12; IM §6.1–6.2, §6.5, §6.10; E02 §01, §08; S03 §01, §03; O05 §02; O10; RTM Ch.3 · elan1 Technology Memorandum · Sept 2026

Figure 3 — The nine planes, P0 to P8, with the owner of each, the planes elan1 owns (P0 policy, P3, P4, P5, P8), and the boundary rules that are enforced in CI or by network policy rather than in review.

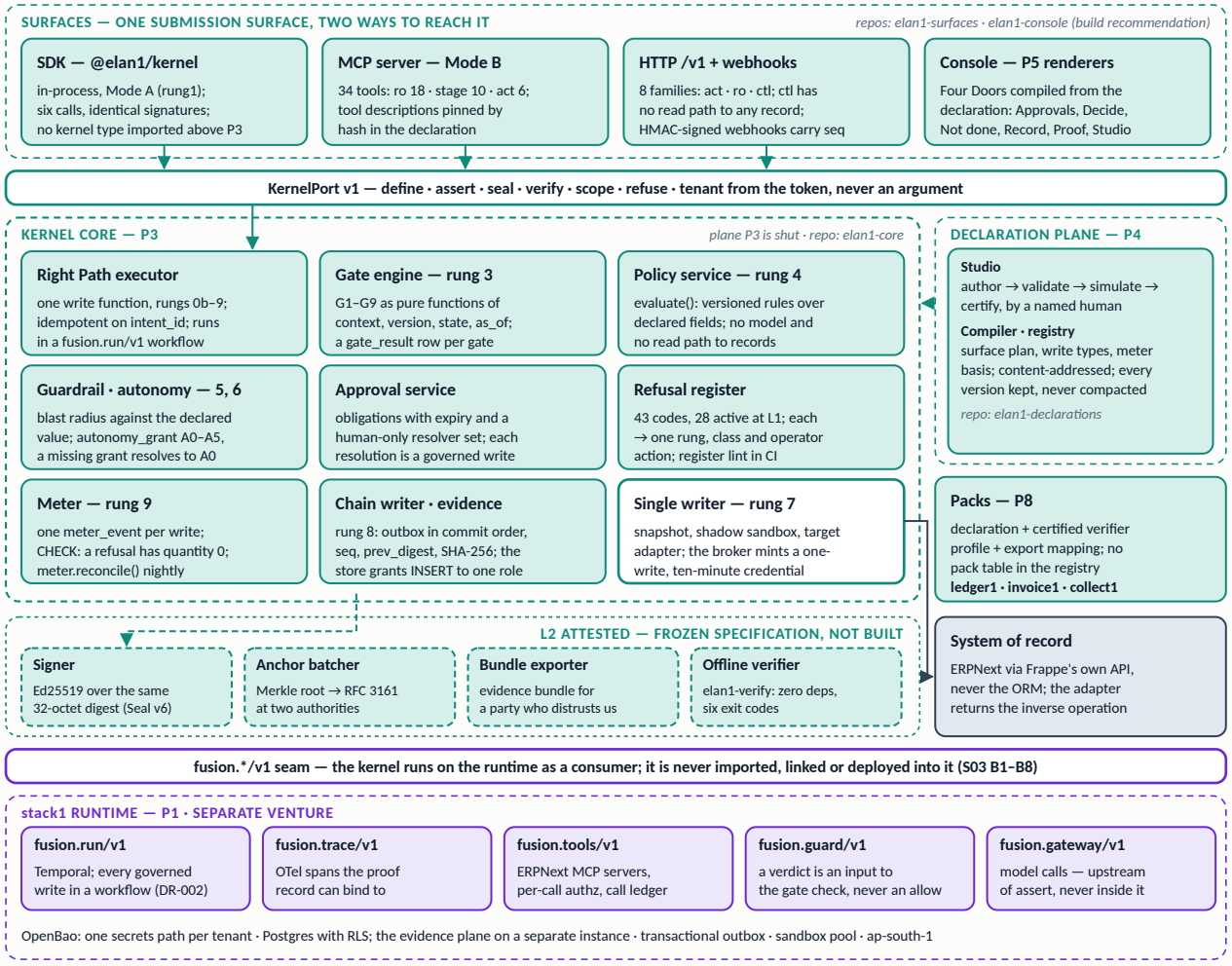
Explained in: 3. Reference architecture — planes, components and repositories (memorandum page 23) · Source file: fig-03-planes.svg

FIGURE 4 OF 32

Figure 4 — Components, repositories and internal seams

L1 — BEING BUILT · L2 — NOT BUILT

What elan1 builds for L1, where each part sits, the frozen L2 parts beside it, and the two seams in and out



Repositories: elan1-core and elan1-declarations are recorded (O03 §12); elan1-surfaces, elan1-console, elan1-packs and elan1-verify are this memorandum's proposal (build recommendation). One writer per file; a pull request crossing two repositories needs both owners.

Every submission lands on one function; the only credential to the books sits at rung 7, and the control plane cannot read a record.

Source: E02 §03–06; E04 §03; E05 §03–09; E06 §01–07; E18 §01; O03 §05–06, §10, §12; O01 DR-002; S02 §02; S03 §01–03; IM §6.9–6.10 · elan1 Technology Memorandum · Sept 2026

Figure 4 — The elan1 components by zone, from the submission surfaces through the kernel stages, single writer, chain writer and meter, with the declaration registry and policy service in the control plane, the Level 2 components drawn as specified-only, and the repository that holds each.

Explained in: 3. Reference architecture — planes, components and repositories (memorandum page 27) · Source file: fig-04-component-map.svg

FIGURE 5 OF 32

Figure 5 — KernelPort v1 — six calls, five objects

L1 — BEING BUILT · [L2] FIELDS FROZEN

The single seam between anything that wants to change a system of record and the thing that decides whether it may

THE SIX CALLS — IDENTICAL SIGNATURES FOR EVERY CONSUMER, AT EVERY ASSURANCE LEVEL

1 · define
register a write type at a declaration version
certified declaration; version must be last + 1
itself a governed write under dual control
idempotent on (declaration_id, version, digest)
never overwrites a version or resolves "latest"

2 · assert
run the rungs and return a decision
act token · the only call that changes a record
returns decision, seq, digest, undo_until (30 d)
idempotent on intent_id: no re-run, no re-meter
never calls a model, caches, or has an admin mode

3 · seal
write the record and chain it (rung 8)
callable only by the kernel: the chain writer
consumes the outbox in commit order, next seq
idempotent on write_id; a gap in seq is a P0
[L2] the same call also signs; shape unchanged

4 · verify
check a record or a chain segment
ro token · recomputes preimage and digest
walks seq from 1; PASS or first failure with
sequence, field and reason · never metered
at L1, verify requires us — said in those words

5 · scope
resolve blast radius and autonomy
ro token · for rendering a screen, or an agent
deciding whether to stage a write at all
advisory: assert re-resolves both, always
cached as permission = the decision cache I5 bans

6 · refuse
emit a typed refusal from the register
kernel, or an adapter before the kernel is reached
code must map to the rung it was raised at
full 16-field record, record_id zeroed, qty 0
never free text, never a 4xx, never a warning

THE FIVE OBJECTS — CONSUMERS HOLD THESE, IDENTIFIERS AND ENUMERATIONS, NEVER A KERNEL INTERNAL

Write intent
a request, never a permission
intent_id — idempotency key
tenant_id — from the token
declaration_id + version
uint32; "latest" is rejected
write_type · channel
actor_id · actor_kind
human | agent | system
on_behalf_of — accountability,
never autonomy
subject · payload
as_of — supplied, never a clock
reason · obligation_ref

Decision
produced by assert; immutable
write_id — kernel-assigned
disposition — allow | refuse |
escalate; there is no fourth
rung · code · gate
obligation — iff escalate
policy_version — resolved by
the kernel, never supplied
autonomy_level — A0–A5
blast_radius — one_record …
cross_boundary
gate_results[] — every gate
reversal_plan

Approval
itself a governed write, not a flag
approval_id · write_id
approver_id — must differ from
actor_id where G4 applies
approver_kind — human only
at L1; an agent never
decision — approve | decline
basis — surface plan version
and rendered payload digest
at — platform clock
decline → the write refuses
POLICY.APPROVAL_DECLINED

Record
the proof object at L1
sixteen fields F01–F16:
tenant_id · seq · prev_digest
write_id · write_type
declaration_id · _version
policy_version · actor_id
actor_kind · autonomy_level
record_id · record_version
payload_hash · lexicon_hash
recorded_at (platform clock)
SHA-256 over LP-encoded
preimage; [L2] Ed25519 signs
the same bytes

Meter event
exactly one per governed write
event_id · write_id · seq
basis — from the declared
blast radius, never per
customer
quantity — 1 committed,
0 every refusal (CHECK)
outcome — committed |
refused | escalated
at — platform clock
every invoice line traces
to a record

THE INVARIANTS — WHERE A CALL AND AN INVARIANT DISAGREE, THE CALL IS A DEFECT

I1	**A model may produce a write intent; it may never produce an allow** rungs 3, 4, 5, 6 and 8 unreachable from any model — import-graph check
I2	Single writer: one process per tenant, single-use ten-minute credential
I3	Fail closed: no grant → A0; missing gate result = failure; outage = platform
I4	Refusals free and typed: one rung, class, operator action; qty 0 by CHECK
I5	No fast path, batch variant, admin mode, decision cache, second implementation
I6	No warn: allow, refuse and escalate are the only outcomes
I7	Nothing resolves to "latest"; every decision records its policy_version
I8	Determinism: same intent, versions and as_of → the same decision forever

Closed refusal register — 43 codes, 28 active at L1
policy 14 · gate 19 · platform 10; the rest activate at L2
each code maps to one rung, one class, one operator action
e.g. GATE.PRECONDITION_FAILED at rung 3 · every PLATFORM.* pages

	refusal	rejection	platform failure
HTTP	200	4xx	5xx, or 200 + code
record	full record	none	full record
meter · retry	qty 0 · never	none · fix	not billed · same id

tenant and actor_kind come from the token (ro · stage · act), never an argument

Six calls, five objects, one writer, one closed refusal register — and no path by which a model can produce an allow.

Source: E02 §01–08 (KernelPort v1, 21 Sep 2026); O01 DR-013; IM §6.3–6.4 · elan1 Technology Memorandum · Sept 2026

Figure 5 — KernelPort v1: six calls, five objects, eight invariants and the closed refusal register, with the token scope each call requires and the one call (assert) that can change a record.

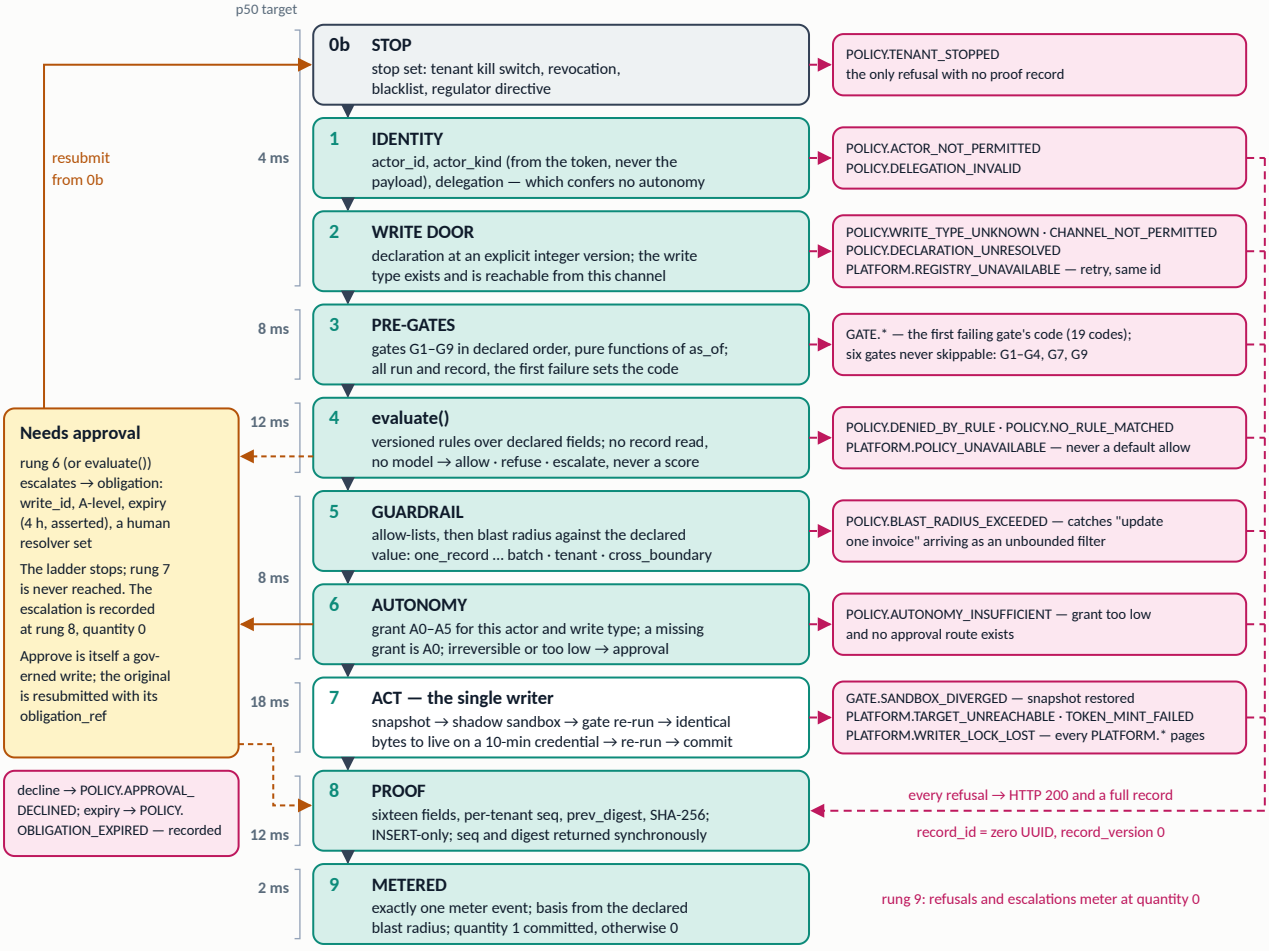
Explained in: 4. The kernel contract — KernelPort, six calls and five objects (memorandum page 40) · Source file: fig-05-kernelport.svg

FIGURE 6 OF 32

Figure 6 — The Right Path — nine rungs in one order

L1 — BEING BUILT · BUDGETS ARE TARGETS

Every governed write climbs the stop check and nine rungs in the same order; every exit is typed, and each leaves a row



What the ladder never has: a model in rungs 3–8 (I1) · a fast path, batch variant, admin mode or decision cache (I5) · a warn (I6)
Rung 8's own failures: PLATFORM.LOG_APPEND_FAILED — the rung-7 transaction already committed, the outbox retries in order, no gap in seq · PLATFORM.CHAIN_HEAD_CONFLICT — two chain writers for one tenant, a P0 that stops the tenant. Every PLATFORM.* code pages.
Kernel total ≈ 64 ms p50 [unverified]; sandbox and live applies sit outside it (≤ 1,200 ms p95 hosted ERPNext; ≤ 2 s p95 end to end, MEASURE THIS).

Every outcome but a stopped tenant writes a record and exactly one meter event; refusals and escalations meter at quantity 0 (CHECK).

Source: E04 §02–04; E02 §02, §05–06; E01 §02; E05 §07; IM §4.3, §6.10 · elan1 Technology Memorandum · Sept 2026

Figure 6 — The Right Path: the stop set at 0b and the nine rungs in fixed order, each with its refusal exit to the proof rung, the escalation exit at rung 6, the sandbox and live applies drawn outside the kernel budget, and the one rung (0b) whose refusal writes no chain entry.

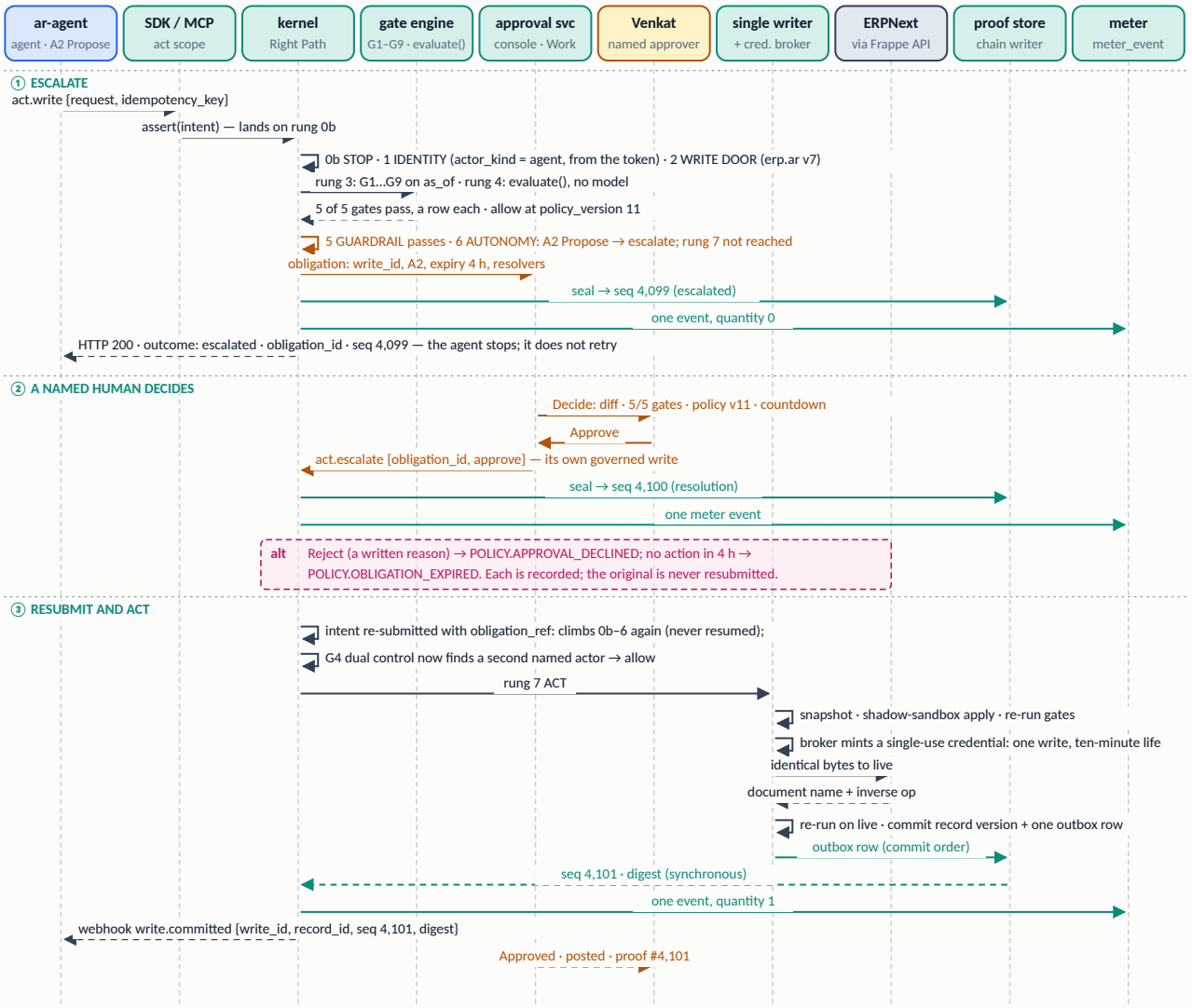
Explained in: 5. The Right Path — nine rungs and the governed action end to end (memorandum page 61) · Source file: fig-06-right-path-rungs.svg

FIGURE 7 OF 32

Figure 7 — One governed action, as a sequence

L1 — DESIGNED, NOT YET BUILT

E04 Flow 3: an agent at A2 proposes a ₹2,40,000 sales invoice, a named human approves, the single writer commits



Three proof entries for one approved decision (a fourth, the lapse, on expiry): the kernel keeps no suspended-and-resumed state.

Source: E04 §03–04 (Flow 3); E02 §03, §05; E05 §04, §08–09; E06 §02–03; IM §4.4 · elan1 Technology Memorandum · Sept 2026

Figure 7 — One governed action end to end: the agent stages and submits through the SDK, the kernel escalates at rung 6, a human approves on the Decide screen (itself a governed write), the resubmission climbs the rungs again, the single writer applies to the shadow sandbox and then to live, and the proof and meter rungs close the action; three chain entries and three meter events result.

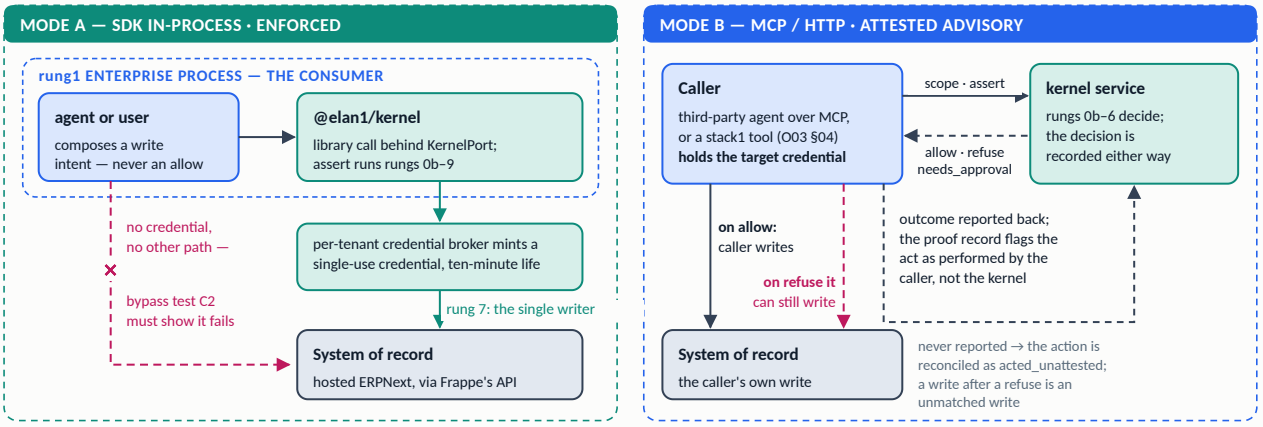
Explained in: 5. The Right Path — nine rungs and the governed action end to end (memorandum page 71) · Source file: fig-07-governed-action-sequence.svg

FIGURE 8 OF 32

Figure 8 — Two modes, two claims

L1 — BEING BUILT · LATENCIES ARE TARGETS

Mode A: the kernel holds the credential and performs the write. Mode B: the caller asks, then writes on its own



	MODE A — ENFORCED	MODE B — ATTESTED ADVISORY
Target credential	the kernel, in the per-tenant credential broker; no long-lived key exists where a consumer, agent or operator can reach it	the caller or its agent — Enforced only where the target accepts kernel-issued tokens (open for ERPNext, O03 OQ1)
Rung 7	performs the write: snapshot, sandbox, identical bytes	does nothing; the caller acts
A refusal means	"the write did not happen"	"elan1 evaluated this and said no"
May claim	*"This write could not have occurred without passing the gates."*	*"Every write intent is evaluated against policy and recorded, and the record is exportable."*
May not say	anything implying L2: "signed", "tamper-proof" (C14)	"we block", "we prevent", "cannot", "nothing gets through without us"
Added latency	≤ 15 ms over the governed-write envelope [target]	≤ 40 ms p95 for the contract call, excluding the target [target]
Kernel fails	the write does not happen; rung1 fails closed: "Nothing was changed."	the caller must fail closed by its own contract; we cannot make it. A timeout on assert is treated as refuse
Proof record	enforcement_mode: enforced — the decision and the act	enforcement_mode: attested_advisory — the decision, plus the caller-performed flag, which must survive into a bundle
Rollback	the kernel's reversal plan, 30 days	the consumer's problem
Phase 1 use	rung1 Enterprise, hosted ERPNext — and nothing else	stack1 tooling; third parties at GA (Phase 2), pre-GA partners

enforcement_mode is set per write type at declaration time, sealed into every record, rendered and exported — never set in a tool call.
Promotion B → A is per write type, never per tenant: 30 days of advisory traffic, a bypass proof, no long-lived credential anywhere, and a rollback drill by both paths; a later write with no record reverts it at once. O03 §04 calls the report-back seal; E02 keeps seal kernel-only.

MCP alone gives advice, not enforcement: a caller that ignores a refusal still writes, and the record says only that it was told not to.

Source: E05 §02–06; O03 §02–04, §08, §13; E01 §05; IM §4.6, §6.9 · elan1 Technology Memorandum · Sept 2026

Figure 8 — Mode A, SDK in-process and Enforced, against Mode B, MCP or HTTP and Attested advisory: where the target credential lives, what rung 7 does, what a refusal means, what the proof record carries, and the one sentence each mode may claim.

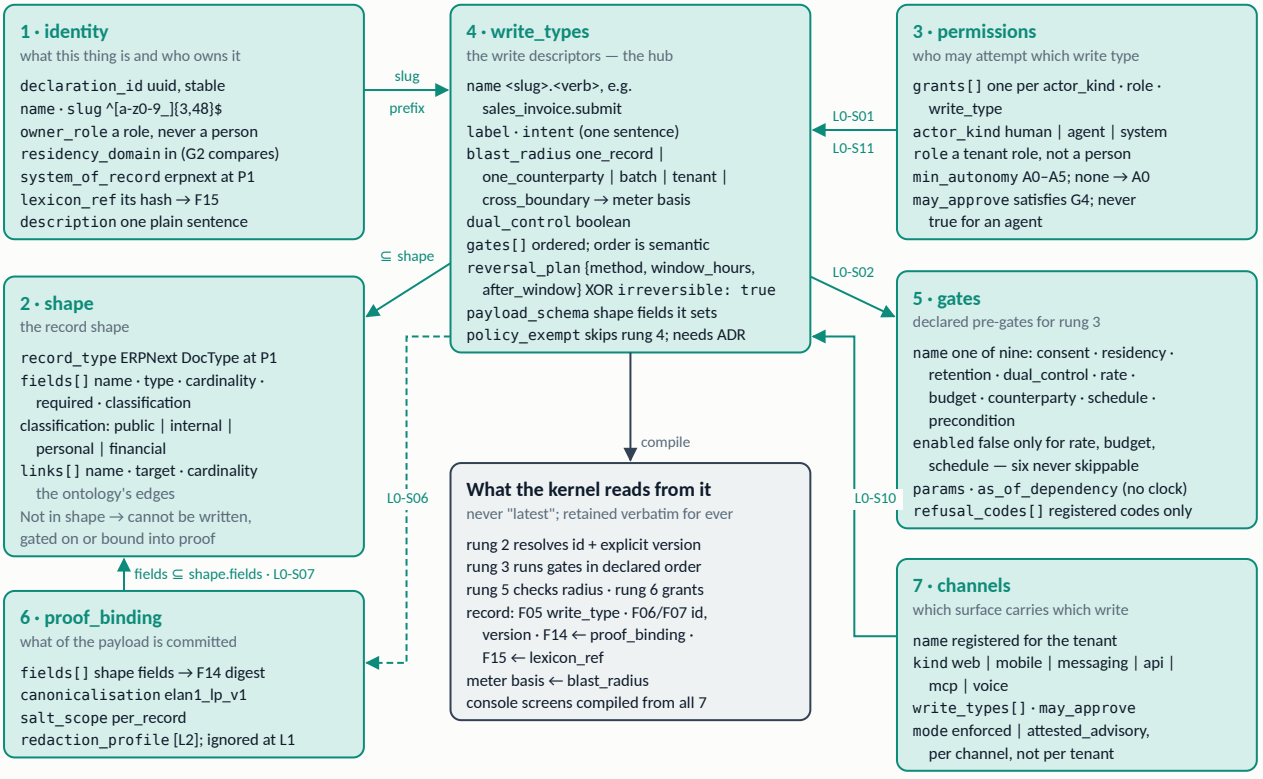
Explained in: 6. Surfaces and enforcement — the SDK, the MCP server and the two modes (memorandum page 80) · Source file: fig-08-enforcement-modes.svg

FIGURE 9 OF 32

Figure 9 — The Declaration — seven blocks and their relations

L1 — BEING BUILT

Seven blocks of one JSON object (under 900 lines); arrows are references checked at VALIDATE, in CI



LIFECYCLE — six stages, none skippable

1 · Author	2 · Validate	3 · Simulate	4 · Certify	5 · Compile	6 · Resolve
seven blocks, Studio or repository file	schema, L0-S01…S11, seven lint warnings	nine rungs, stub writer, reference tenant only	named human; a governed write, dual_control	deterministic, content-addressed; 2 digests = P0	rung 2: id + integer version, never "latest"

Eleven semantic rules at VALIDATE, in CI, with no platform

- L0-S01 granted write types exist
- L0-S02 named gates exist in gates
- L0-S03 reversal_plan XOR irreversible [1]
- L0-S04 irreversible ⇒ ≥ one_counterparty [1]
- L0-S05 tenant / cross_boundary ⇒ dual [1]
- L0-S06 cross_boundary ⇒ proof_binding
- L0-S07 bound fields exist in shape
- L0-S08 channels registered for tenant
- L0-S09 policy_exempt needs ADR
- L0-S10 every write type on ≥ 1 channel
- L0-S11 no agent may_approve; A0–A5

[1] mirrored as database constraints

Three dispositions — never a blank

a value of the declared type
out_of_scope + ADR-nnnn reference
not_measurable + reason + would_change_if (each ≥ 12 chars)
Phase 4 (E21): Control and Regulation families bind write types by name + version

Policy is data: a new regime needs a declaration, gate parameters, a profile and an approver model — not new kernel code.

Source: E03 §02–07, E21 §02–03, IM §4.5, §6.6 · elan1 Technology Memorandum · Sept 2026

Figure 9 — The seven Declaration blocks, the references between them that validation resolves, and the Controls and Regulations families that bind to them by name and version.

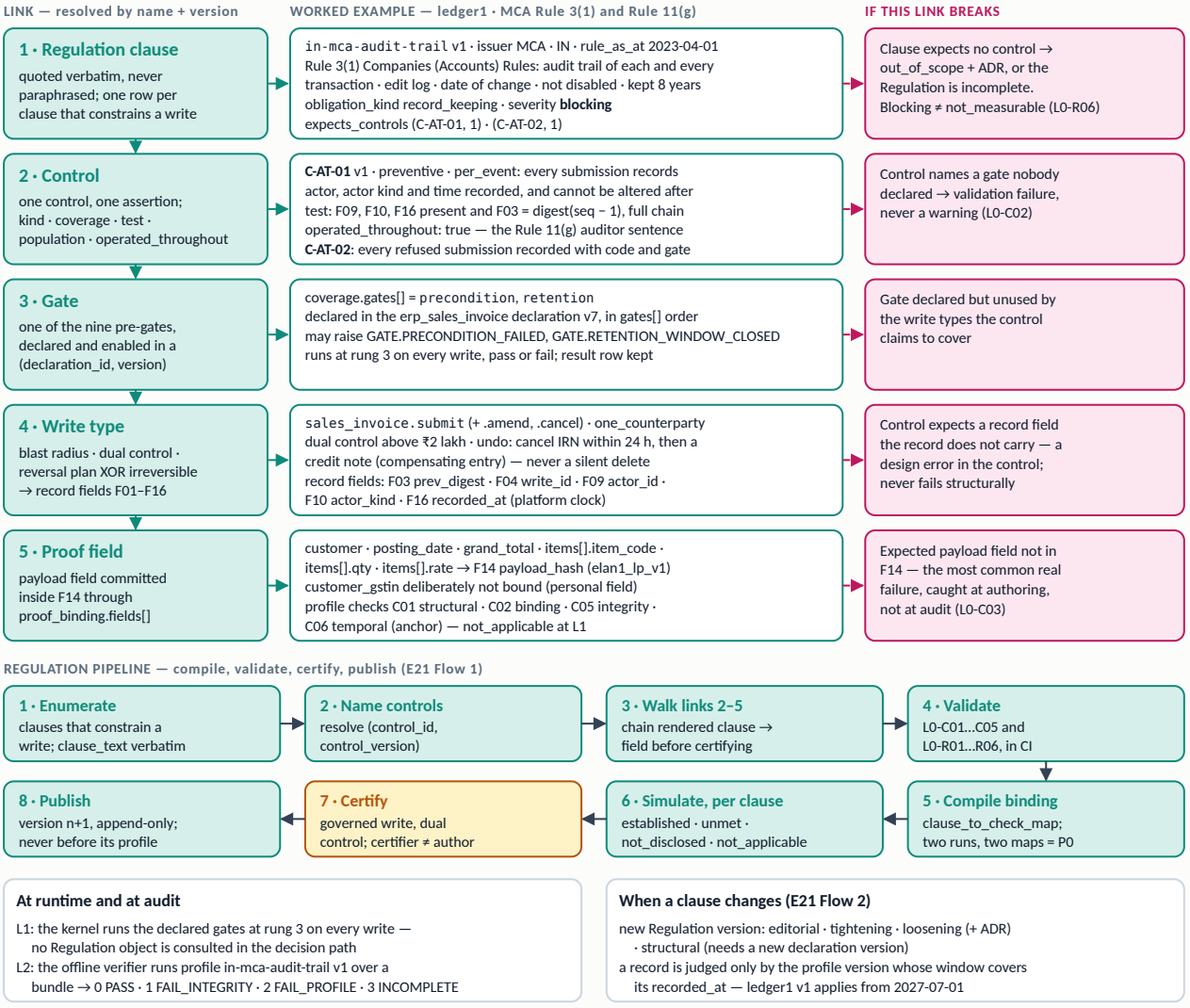
Explained in: 7. The Declaration — the ontology, controls, regulations and policy as data (memorandum page 93) · Source file: fig-09-declaration-ontology.svg

FIGURE 10 OF 32

Figure 10 — From clause to proof field — the five-link chain

PHASE 4 SPEC — NOT BUILT

Every link resolves by name and version at VALIDATE, in CI; a broken link has a name and fails the build



IM §6.7 draws the same chain on journal_entry.submit (batch, dual control always) — another ledger1 write type [E18 §06].

Source: E21 §03–08, E13 §07, E18 §06, E03 §04, IM §6.7 · elan1 Technology Memorandum · Sept 2026

Figure 10 — The five-link chain from a regulation clause to a proof field, each link resolved by name and version in CI, worked for the MCA audit-trail rule down to a runtime gate and a hash-chained record.

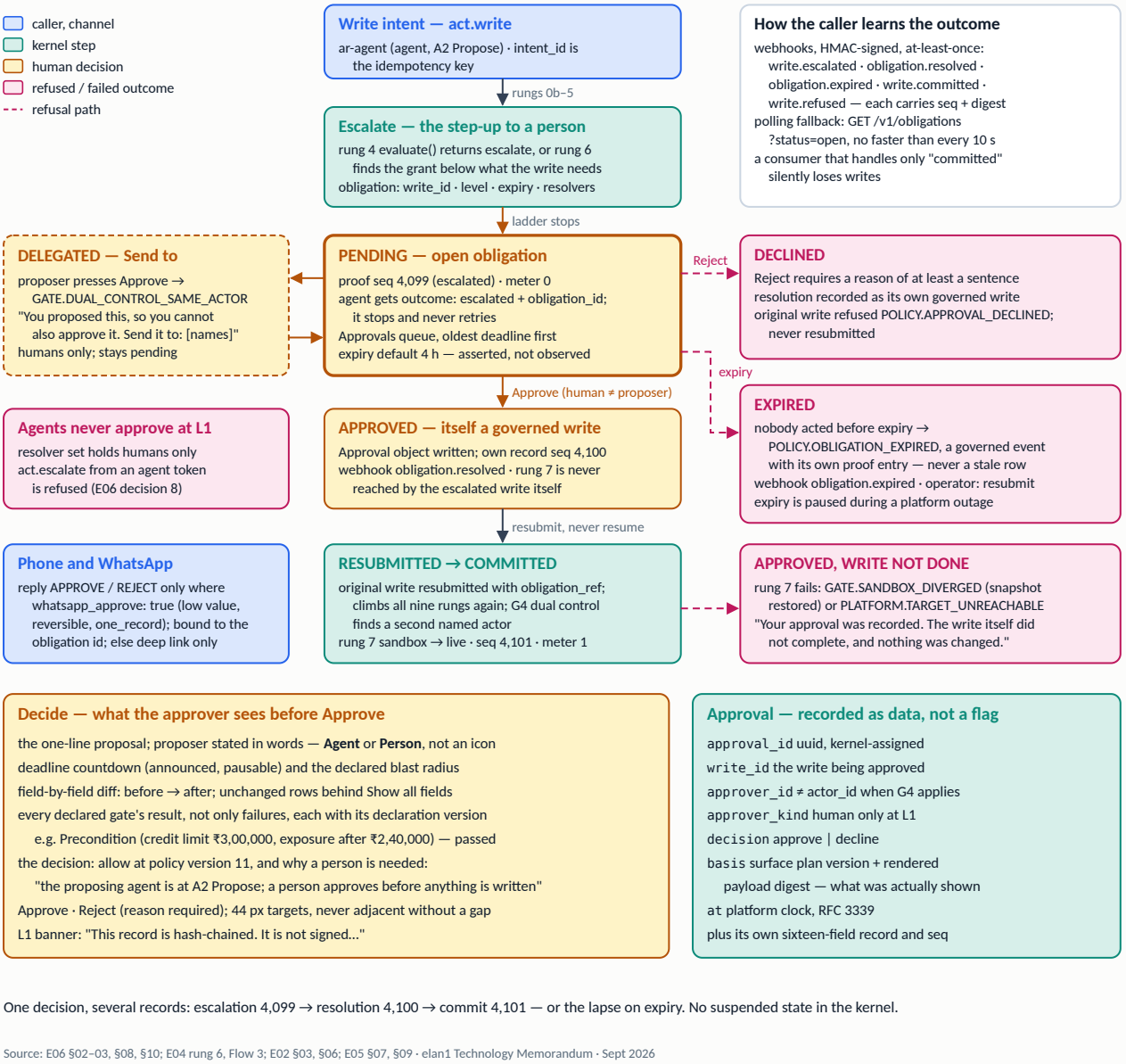
Explained in: 7. The Declaration — the ontology, controls, regulations and policy as data (memorandum page 105) · Source file: fig-10-regulation-compile-chain.svg

FIGURE 11 OF 32

Figure 11 — The approval path — obligation state machine

L1 — BEING BUILT

An escalated write stops; a named human resolves it; every outcome, including expiry, is its own record



One decision, several records: escalation 4,099 → resolution 4,100 → commit 4,101 — or the lapse on expiry. No suspended state in the kernel.

Source: E06 §02–03, §08, §10; E04 rung 6, Flow 3; E02 §03, §06; E05 §07, §09 · elan1 Technology Memorandum · Sept 2026

Figure 11 — The approval path: an escalation at rung 4 or 6 opens an obligation; resolution is itself a governed write; approval causes a resubmission through all nine rungs; decline and expiry are governed events with their own proof entries.

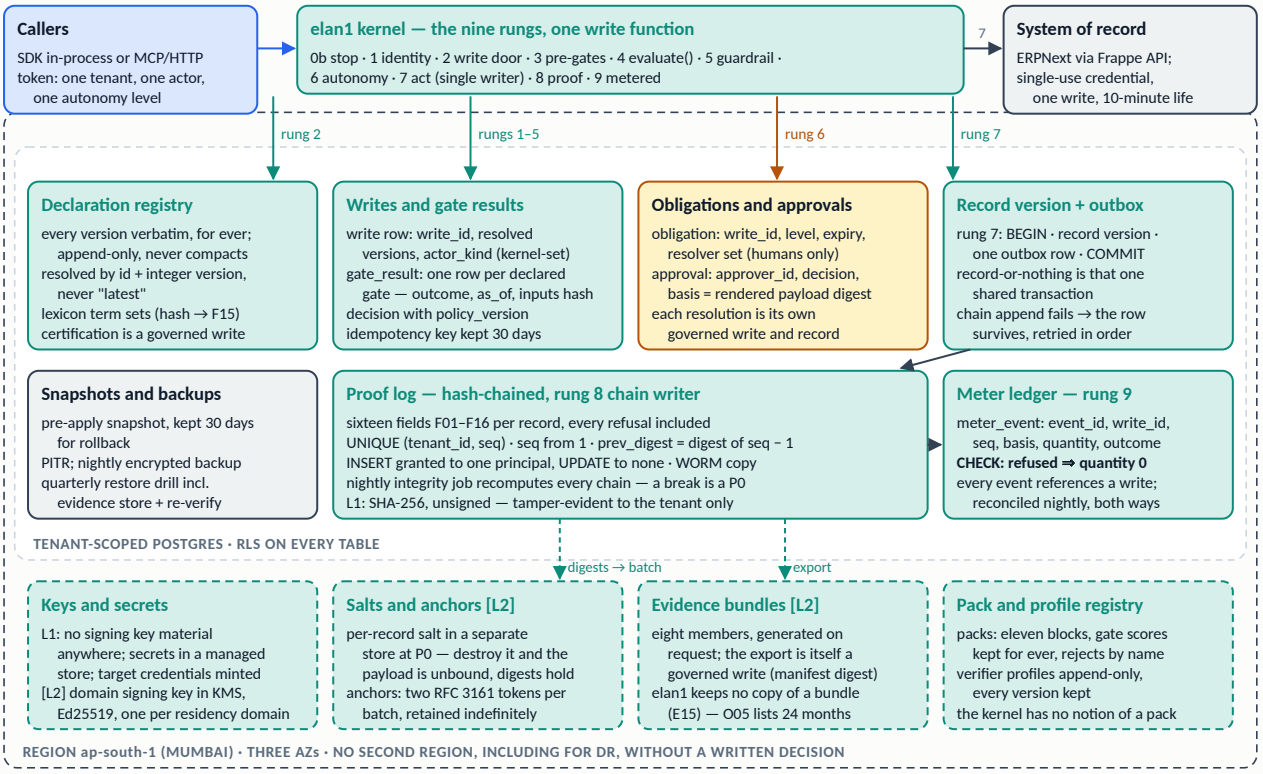
Explained in: 8. The console and the human approval path (memorandum page 117) · Source file: fig-11-approval-flow.svg

FIGURE 12 OF 32

Figure 12 — Data architecture — stores, flows, tenancy and retention

L1 — BEING BUILT · L2 DASHED

Every store sits in one Indian region; every tenant row carries tenant_id under row-level security



Retention by class

Proof records (the chain)	MCA Rule 3(1): ≥ 8 years · O05 table: 7 years WORM **(conflict)**
	E11 / E15: chains and anchors held indefinitely
Declarations, profiles, lexicons	verbatim, for ever — the 2027 screen rebuilt in 2035
Anchors [L2]	indefinitely, with TSA certificate path + revocation data
Evidence bundles [L2]	O05: 24 months WORM · E15: not retained by elan1
Snapshots · backups	30 days, encrypted, in India
Security and access logs	≥ 1 year; 180-day minimum in India (CERT-In)
Application logs	180 days, append-only, no PII (CI lint)
Idempotency keys	30 days

Tenancy and residency, checked in CI

RLS on every tenant table; a migration that disables RLS fails CI
cross-tenant read test runs in CI, expects failure
a call not attributable to a tenant is rejected
caches, indexes, vector stores partitioned by tenant, not filtered at query time
control plane: no network route to records
Postgres primary + synchronous standby (2nd AZ)
CI fails any resource outside the region list
residency and key custody at plane P0: no migration path

At L1 every store is checkable only by those who trust elan1; E11 test 3 — reconciliation to the target system — is the outside check.

Source: E02 §03, §05; E04 rungs 7–9; E11 §2; E07; E15 §04; E16; O05 §1–6; IM §6.10, §7.1 · elan1 Technology Memorandum · Sept 2026

Figure 12 — elan1 data stores in one residency domain: the declaration registry, the action store, approvals, the proof log, the meter ledger, the salt and key stores and the pack catalogue, each with its single writer, its tenancy boundary and its retention class.

Explained in: 9. Data architecture and the Level 1 proof record (memorandum page 127) · Source file: fig-12-data-architecture.svg

FIGURE 13 OF 32

Figure 13 — The Level 1 proof record — a per-tenant hash chain

L1 — BEING BUILT

Sixteen frozen fields, SHA-256 over a length-prefixed preimage, each record carrying its predecessor's digest

One record — sixteen fields

fixed order · encoded octets

F01	tenant_id	uuid · 16
F02	**seq**	uint64 · 8
F03	**prev_digest**	digest · 32
F04	write_id	uuid · 16
F05	write_type	string · 5–64
F06	declaration_id	uuid · 16
F07	declaration_version	uint32 · 4
F08	policy_version	uint32 · 4
F09	actor_id	uuid · 16
F10	actor_kind	enum · 5 or 6
F11	autonomy_level	uint8 · 1
F12	record_id	uuid · 16
F13	record_version	uint32 · 4
F14	payload_hash	digest · 32
F15	lexicon_hash	digest · 32
F16	recorded_at	timestamp · 20

seq 1 — genesis (TV-001)

F02 seq = 1
F03 prev_digest = 32 × 0x00
F05 sales_invoice.submit
F10 human · F11 2
F12 record id · F13 1
F16 2027-01-04T09:15:00Z
preimage exactly 329 octets
digest d_1 = SHA-256(preimage)

…

seq 4,097 — committed

F03 prev_digest = d_{4096}
F05 erp.payment_entry.submit
F07 decl. v7 · F08 policy 11
F10 human · F11 4 (A4)
F12 ERPNext record id
F13 record version ≥ 1
F14 salted payload digest
digest d_{4097} · meter 1

link

seq 4,098 — refused

F03 prev_digest = d_{4097}
F05 erp.sales_invoice.submit
F10 agent · F11 3 (A3)
F12 zero UUID · F13 0
same sixteen fields, same shape as a commit
GATE.PRECONDITION_FAILED
digest d_{4098} · meter 0

Kept beside each record, outside the preimage: a gate_result row per declared gate, the refusal code, the reversal plan, one meter event — and the approval, which is its own record

The preimage — no JSON, length prefixes, no optional fields

`preimage = LP("elan1.proof.seal.v6") ‖ LP(enc(F01)) ‖ … ‖ LP(enc(F16))`
`LP(x) = uint32_BE(len(x)) ‖ x` · 17 groups · `digest = SHA-256(preimage)`
L1: no signature, key_id or anchor_ref member — absent, not null or stubbed · assurance_level: 1

Writing the chain — rung 8, one writer per tenant

chain writer consumes the outbox in commit order, not arrival order
assigns the tenant's next seq; reads prev_digest from seq − 1
binds the sixteen fields from the append-only write context
appends under UNIQUE (tenant_id, seq); INSERT to one principal, UPDATE to none; returns seq + digest synchronously
LOG_APPEND_FAILED outbox row survives; retried in order, never skipped
CHAIN_HEAD_CONFLICT two writers for one tenant — P0, tenant stops
tenant stop (rung 0b) is the only refusal with no record

Detecting a gap — verify and the nightly job

recompute every preimage and digest, octet for octet
walk seq contiguous from 1; each prev_digest = prior digest
resolve every declaration version and lexicon hash
report PASS or the first failure: seq, field, reason
nightly job re-walks every tenant's whole chain; a gap is a P0
continuity is three-valued: continuous · break at seq N · not checked over this range — scope and count printed
cannot see a cut tail: 4,000 of 4,200 looks complete

What an L1 record establishes

a change of a stated type, by a named actor of a stated kind, at a stated position in an ordered sequence
which rule versions were in force — resolvable verbatim
which gates ran, in order, and what each returned
who approved, and a digest of what they were shown
applied once, not twice; the refused cases are present
nothing retained was altered since written — to anyone willing to trust elan1's store: tamper-evident

What it does not establish

that the rules were the right rules
that the underlying transaction was genuine
the target's current state — a direct database edit afterwards leaves the chain reading clean
anything to a party who distrusts elan1: whoever holds the store could recompute the whole chain
a truncated tail — test by reconciling to the target (test 3)
any opinion or conformance; never "tamper-proof"

L2 signs these exact bytes; records written at L1 stay L1 and are never back-signed.

Source: E02 §03, §05 (seal, verify); E04 rung 8, Flows 1–2; E12 §02–05, §07; E11 §2–6; E07; IM §7.1 · elan1 Technology Memorandum · Sept 2026

Figure 13 — The Level 1 proof chain: each record carries sixteen fields including the digest of its predecessor; genesis carries thirty-two zero octets; refusals carry the zero record identifier; the chain writer appends under a unique (tenant, seq) key and the nightly job recomputes every link.

Explained in: 9. Data architecture and the Level 1 proof record (memorandum page 134) · Source file: fig-13-l1-proof-chain.svg

FIGURE 14 OF 32

Figure 14 — Seal Spec v6 — sixteen fields to a signed, anchored seal

L2 — SPECIFIED, NOT BUILT

elan1.proof.seal.v6 is the one hard freeze: L1 chains these bytes now, L2 signs and anchors the same bytes

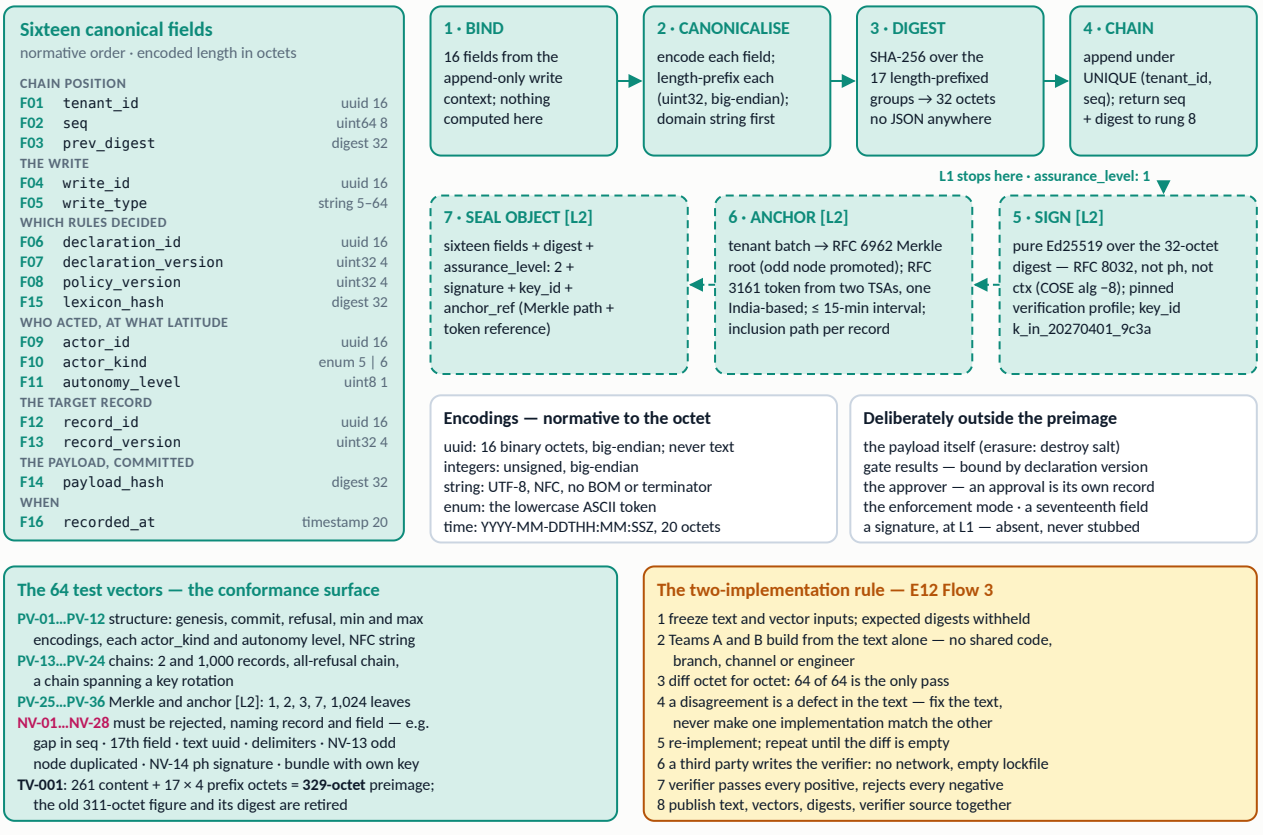


Figure 14 — Seal v6: sixteen fields are encoded and length-prefixed behind the domain string, hashed with SHA-256, and — at Level 2 only — signed with pure Ed25519 and anchored through a Merkle root at two RFC 3161 authorities; no record leaves the building until two independent implementations agree on all sixty-four vectors.

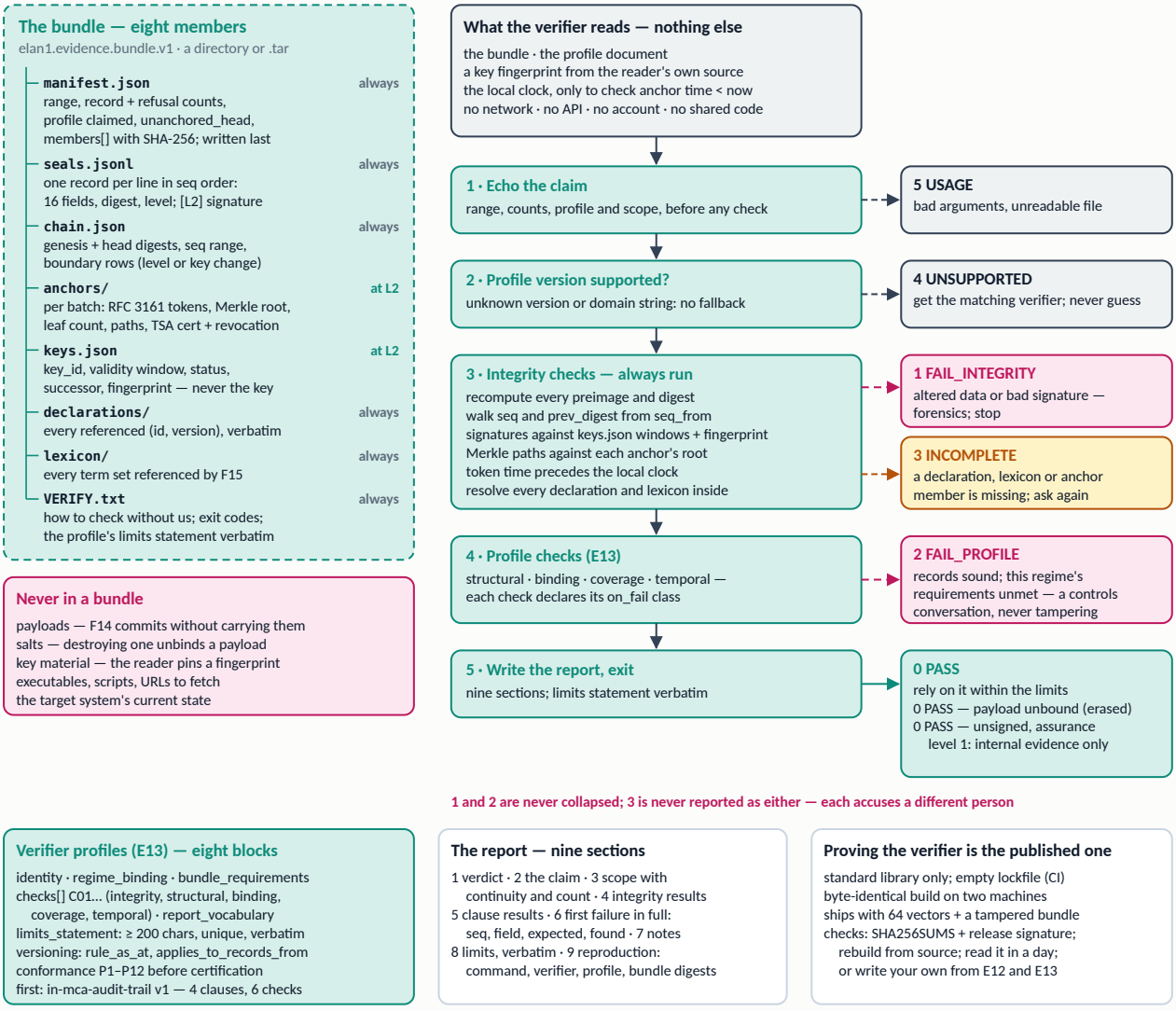
Explained in: 10. Seal Spec v6 — canonical fields, signatures and test vectors (memorandum page 150) · Source file: fig-14-seal-v6.svg

FIGURE 15 OF 32

Figure 15 — The evidence bundle and the offline verifier

L2 — SPECIFIED, NOT BUILT

A bundle needs nothing from elan1; the verifier reads it on an air-gapped machine and answers with one of six codes



A verifier nobody has watched refuse is not a verifier: run it on the published tampered bundle and watch it exit 1.

Source: E15 §01–08; E13 §02–07; E12 §07; IM §7.4–7.5, §7.8 · elan1 Technology Memorandum · Sept 2026

Figure 15 — The evidence bundle and the offline verifier: eight inert members bound by the manifest; the verifier and the key fingerprint arrive from the reader's own sources; integrity checks always run, profile checks run against the claimed profile, and the run ends in one of six exit codes that never collapse integrity, profile and incompleteness into one.

Explained in: 11. The evidence bundle, the offline verifier and verifier profiles (memorandum page 165) · Source file: fig-15-evidence-bundle-verifier.svg

FIGURE 16 OF 32

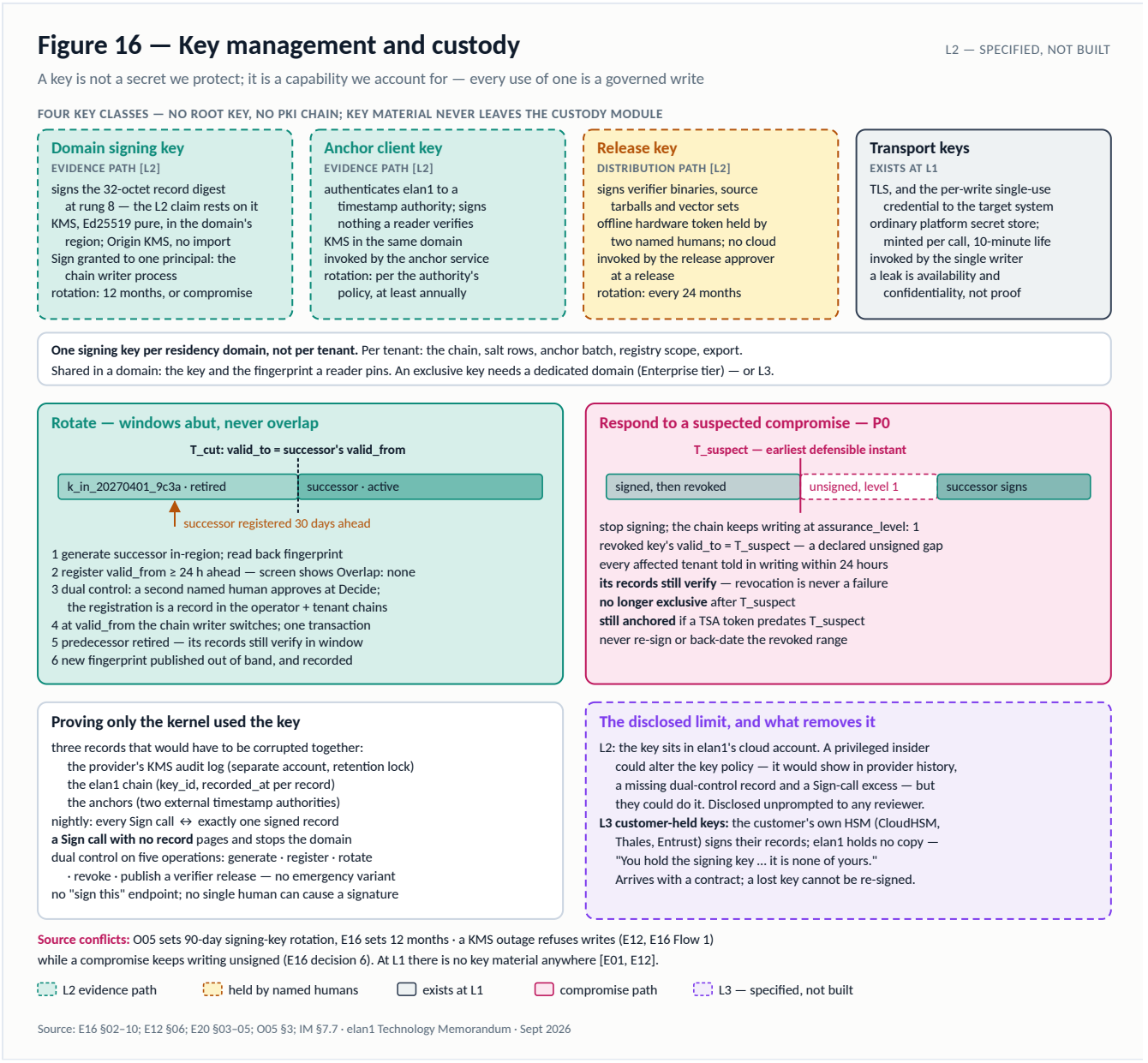


Figure 16 — Key classes by residency domain: the domain signing key and anchor client key in a managed KMS invoked only by the chain writer and anchor service, the offline release key, abutting rotation windows, three independent records of key use, and the insider limitation that customer-held keys remove at L3.

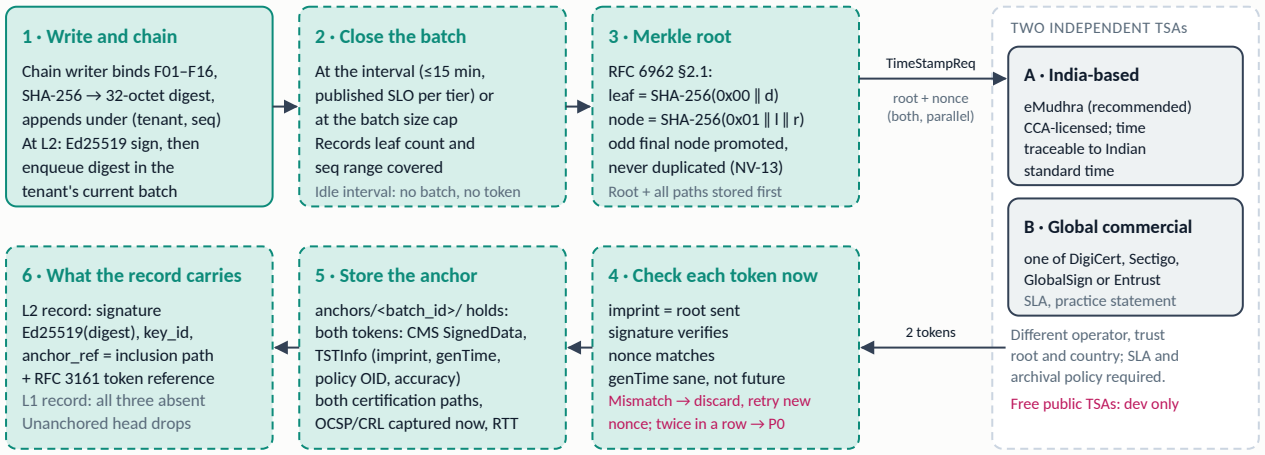
Explained in: 12. Key management, custody and timestamp authorities (memorandum page 177) · Source file: fig-16-key-custody.svg

FIGURE 17 OF 32

Figure 17 — Two-authority timestamp anchoring

L2 — SPECIFIED, NOT BUILT

A per-tenant batch root goes to two independent RFC 3161 authorities; anchoring never refuses or delays a write



Flow 2 — an authority is unreachable

1 Retry it with backoff for the rest of the interval; if the other answered, the batch is marked single_anchor (degraded, not failed)
2 Next batch opens on schedule; batches are never held open
3 Same TSA failing 2 consecutive intervals → SRE paged
4 Both failing 4 consecutive intervals → Assurance door banner: "Anchoring degraded since 09:14 — 88 records unanchored"
5 On return: backlog anchored oldest first, each with its own root, never merged; true token time kept, gap left visible
6 TSA whose tokens fail our checks → removed; run with one
7 Two tokens disagree about the root → integrity failure, P0

Anchoring is not a gate: no refusal path, never blocks a write

Offline verification (evidence bundle)

Recompute leaf → inclusion path → root; root = imprint
Token signature verifies against the TSA certificate
Certification path reaches a trust anchor
genTime precedes the local clock (weak by design)
Bundle verifies if either token verifies
Exit 1 FAIL_INTEGRITY · path or anchor fails
Exit 3 INCOMPLETE · anchor member missing
Level 1 records: anchor checks not_applicable
Long-term: validity assessed as at genTime from the captured path + OCSP/CRL; a TSA ceasing leaves tokens checkable; a distrusted root → the other TSA carries it

What an anchor proves

This digest existed, in this form, no later than genTime, per a party with no interest in the matter: existence, form, an upper bound on time.

Catches a chain cut at the tail (leaf count) and a chain recomputed wholesale with plausible F16 recorded_at values.

What it does not prove

· that the record was created at that time
· completeness: records never submitted are invisible
· that we are honest, or that the record is true
· any legal effect (not our determination)

Re-anchoring later adds a later bound, never replaces one

elan1 kernel (L1) · L2 anchor service · external authority / verifier · failure handling

Anchoring is machinery, not a governed write: not in the tenant chain; a fixed cost in the L2 add-on, never a metered unit.

Source: E17 §01–§09, E12 §04–§05, E15 §02, §06, E16 §02 · elan1 Technology Memorandum · Sept 2026

Figure 17 — Two-authority anchoring: per-tenant batch, RFC 6962 Merkle root with odd node promoted, parallel RFC 3161 requests to an India-based and a global authority, token checks, anchors/ contents, and the single_anchor, paging and banner states when an authority fails.

Explained in: 12. Key management, custody and timestamp authorities (memorandum page 188) · Source file: fig-17-timestamp-anchoring.svg

FIGURE 18 OF 32

Figure 18 — The pack factory: five gates, eleven blocks, eight tests

DESIGNED — NOT YET BUILT

A pack is configuration, not code: a candidate must pass all five admission gates before any build work starts

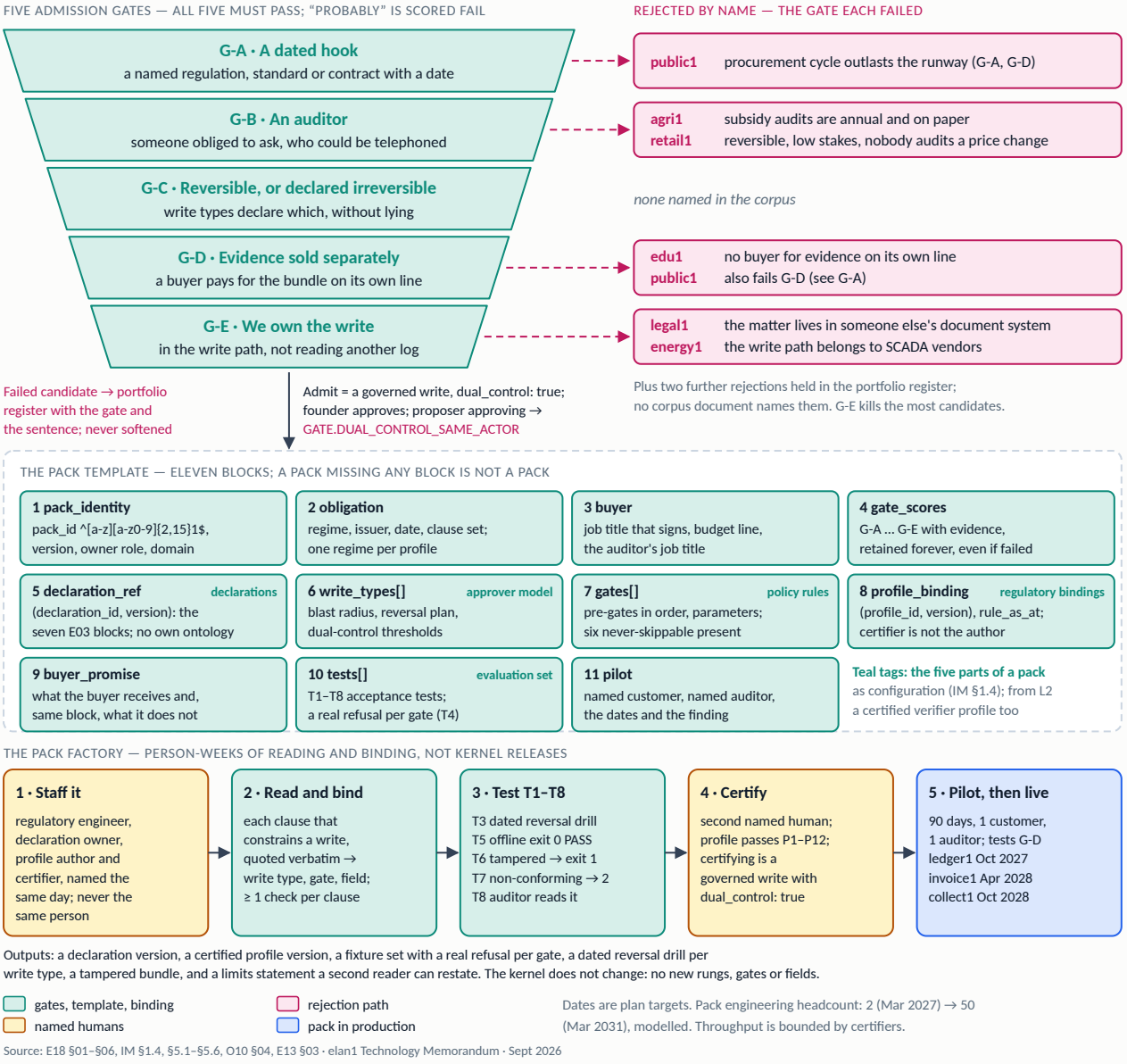


Figure 18 — The pack factory: a candidate is scored against the five gates G-A to G-E (eight rejected by name), admitted by a governed write under dual control, composed from the eleven-block template, tested against T1–T8, piloted with a named auditor and certified.

Explained in: 13. Packs — the pack factory, ledger1, invoice1 and collect1 (memorandum page 197) · Source file: fig-18-pack-factory.svg

FIGURE 19 OF 32

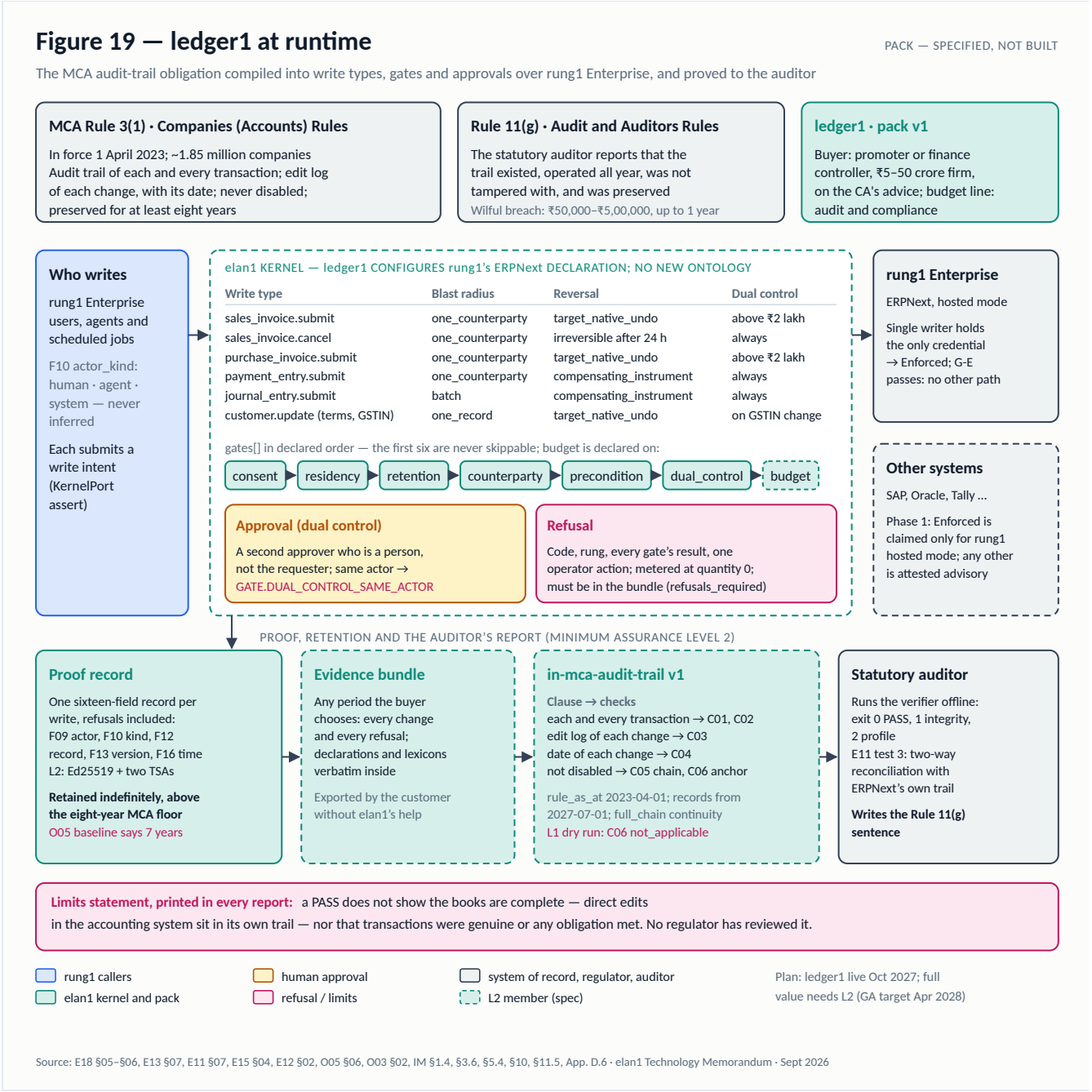


Figure 19 — ledger1 at runtime: a books-of-account write under rung1 Enterprise passes the nine rungs with the audit-trail precondition, is approved under dual control, is applied through Frappe's API by the single writer and hash-chained; at L2 the chain is signed, anchored and exported to the statutory auditor, who verifies offline and reconciles against ERPNext's own trail.

Explained in: 13. Packs — the pack factory, ledger1, invoice1 and collect1 (memorandum page 203) · Source file: fig-19-ledger1-runtime.svg

FIGURE 20 OF 32

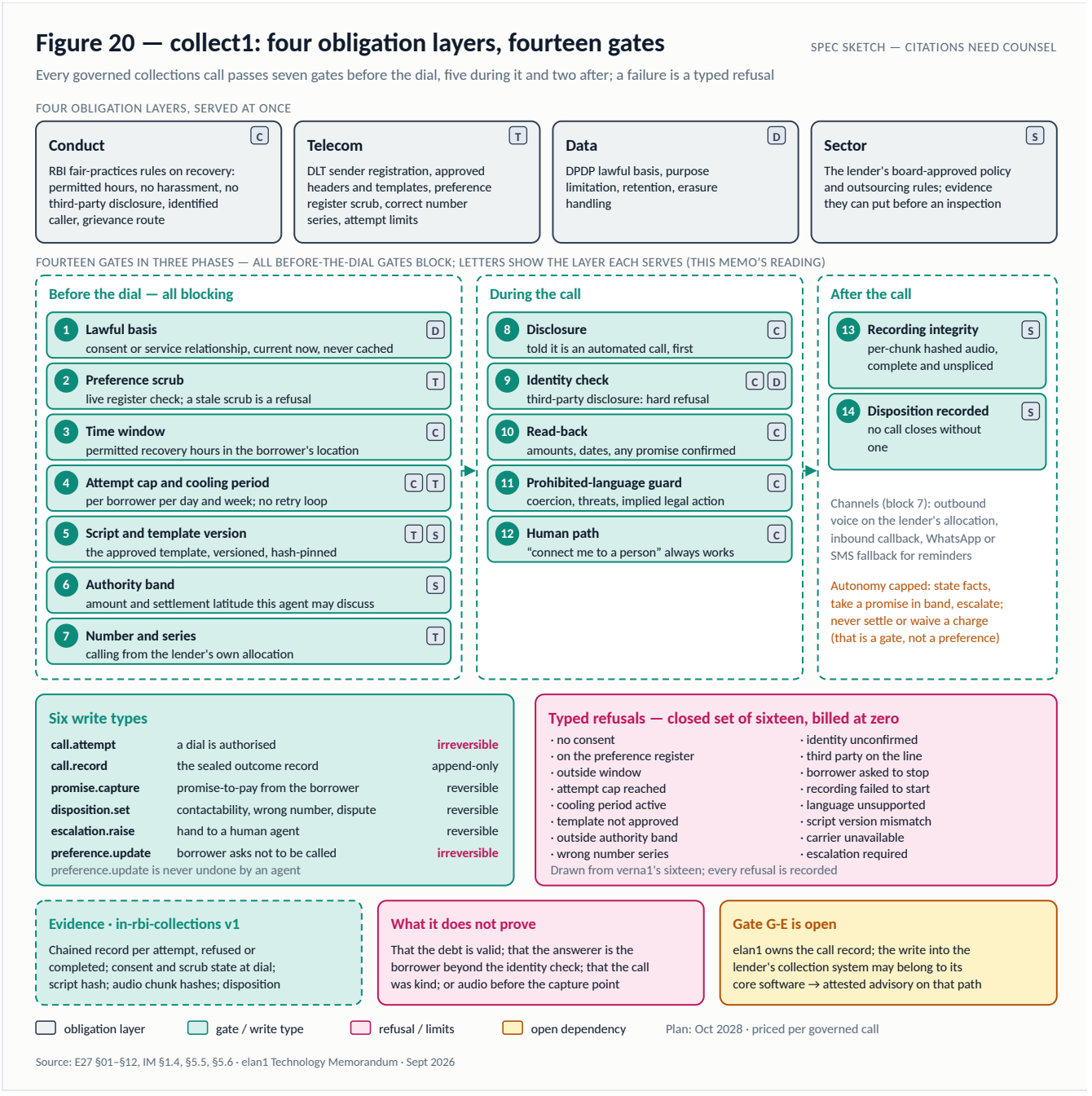


Figure 20 — collect1: four obligation layers (conduct, telecom, data, sector) and fourteen gates in three phases — seven before the dial, five during the call, two after — mapped onto the kernel's nine pre-gates, rung 4 rules and the preconditions of the writes that record the call.

Explained in: 13. Packs — the pack factory, ledger1, invoice1 and collect1 (memorandum page 207) · Source file: fig-20-collect1-layers.svg

FIGURE 21 OF 32

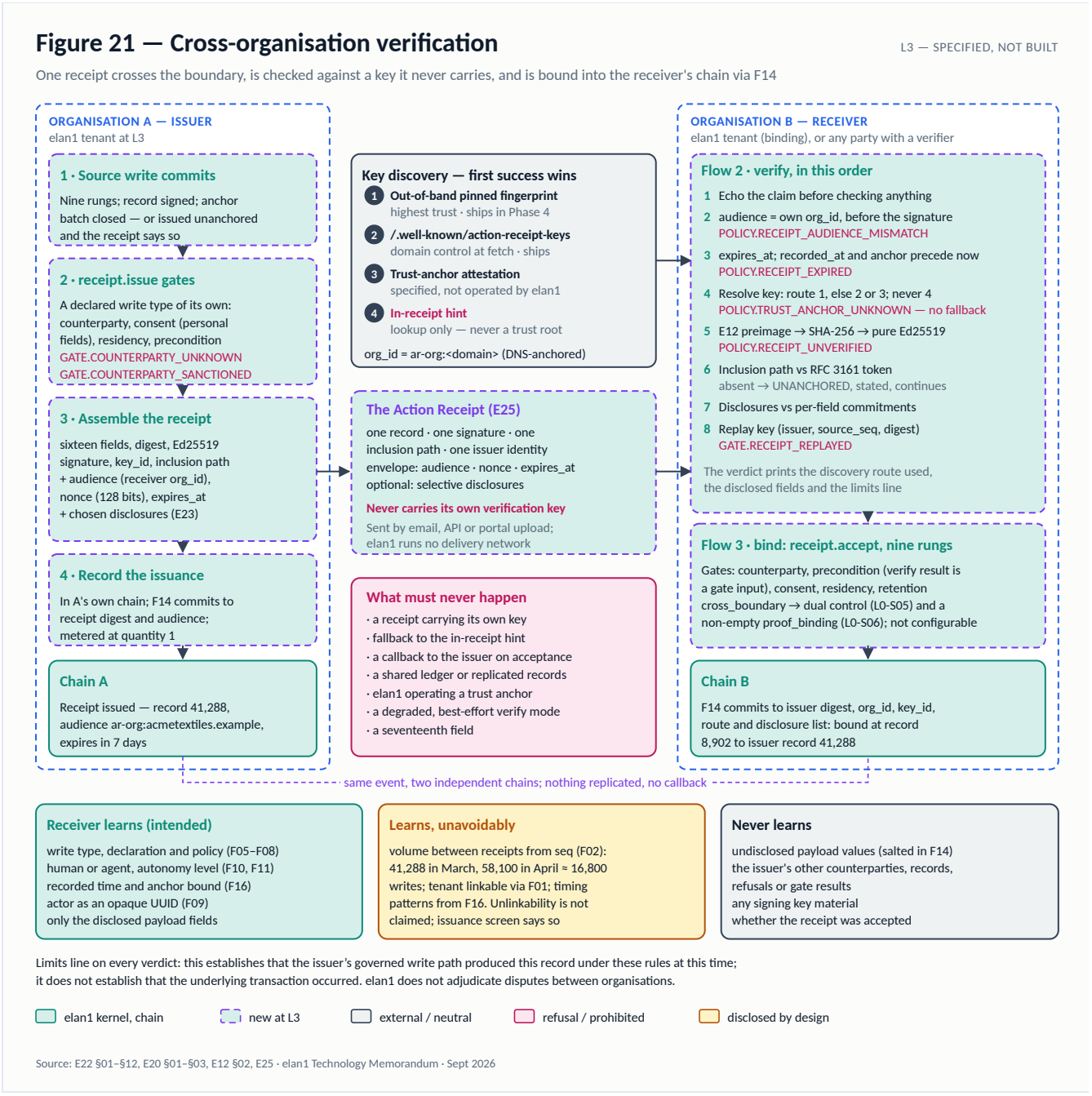


Figure 21 — Cross-organisation verification: the issuer's governed write and receipt.issue record, the receipt (sixteen fields, signature, inclusion path, audience, nonce, expiry) travelling by the business's own channel, key discovery by pinned fingerprint, issuer key document or trust anchor, offline verification, and the receiver's receipt.accept record binding the issuer's digest through F14.

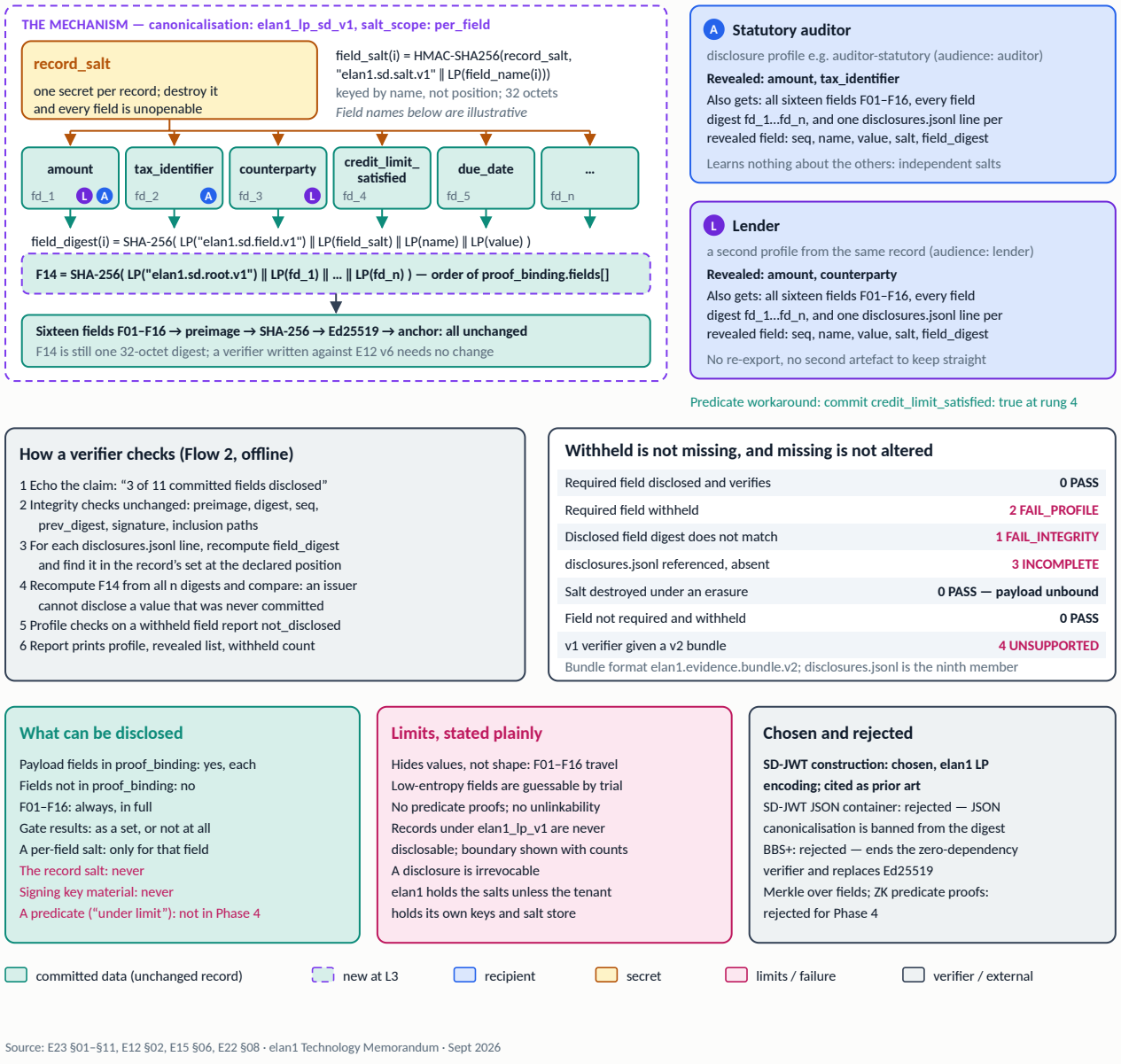
Explained in: 14. Level 3 — cross-organisation verification, selective disclosure and earned autonomy (memorandum page 216) · Source file: fig-21-cross-org-verification.svg

FIGURE 22 OF 32

Figure 22 — Selective disclosure: revealed versus committed

L3 — SPECIFIED, NOT BUILT

Per-field salted commitments fold into F14, so one record reveals different fields to different recipients



Source: E23 §01–§11, E12 §02, E15 §06, E22 §08 · elan1 Technology Memorandum · Sept 2026

Figure 22 — Selective disclosure under elan1_lp_sd_v1: per-field salts derived from the record salt, per-field digests folded into F14, the unchanged sixteen fields and signature, and two recipients each receiving different openings in disclosures.jsonl while the verifier recomputes F14 from disclosed and carried digests.

Explained in: 14. Level 3 — cross-organisation verification, selective disclosure and earned autonomy (memorandum page 222) · Source file: fig-22-selective-disclosure.svg

FIGURE 23 OF 32

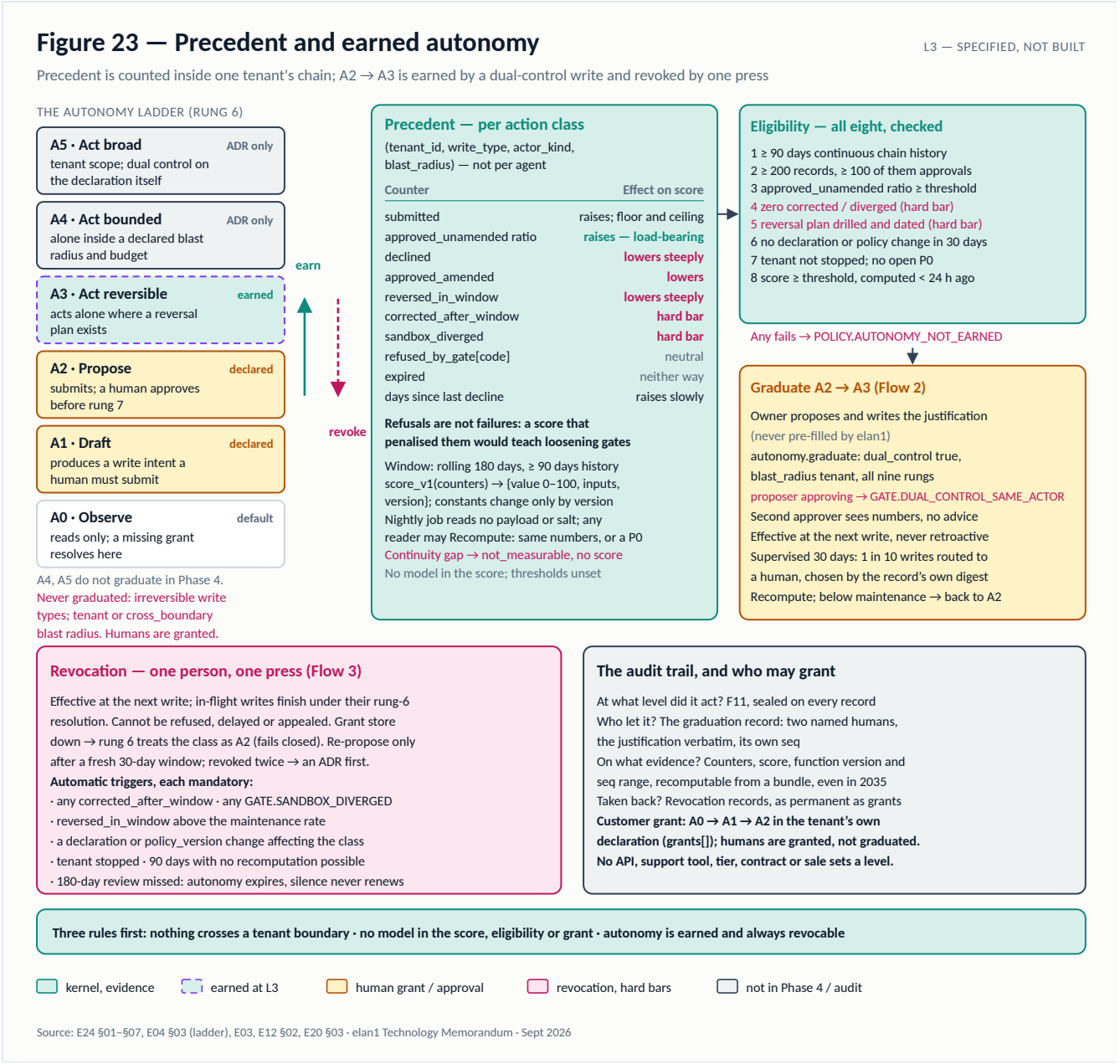


Figure 23 — Earned autonomy: the A0–A5 ladder resolved at rung 6, precedent counters projected from the tenant's own chain into a deterministic versioned score, the eight eligibility conditions, dual-control graduation from A2 to A3 with a 30-day supervised period, and one-person revocation with its automatic triggers.

Explained in: 14. Level 3 — cross-organisation verification, selective disclosure and earned autonomy (memorandum page 227) · Source file: fig-23-earned-autonomy.svg

FIGURE 24 OF 32

Figure 24 — Action Receipt standard, certification and regulators

PHASE 4 — SPECIFIED, NOT BUILT

Publish the format and a free verifier, keep the write path; certify only when a paying customer's auditor names a standard

Given away, permanently

Specification text — CC BY 4.0
Reference verifier — Apache-2.0, never priced, to anyone
Test vectors, tampered bundle — CC0
Verification procedure and the suite registry
Errata, disagreements, defects

Kept and sold

The kernel: gates as data, forced approval, the single writer
Packs and certified profiles
Metered governed action; levels
Residency, custody, anchoring, retention, the evidence SKU
Cross-org verification seats

The name "elan1" and its marks are not licensed

Why publishing is safe: a receipt attests only a call that was made.
The single writer makes a missing receipt itself evidence — the suppression attack no format solves is closed by the architecture

Receipt members — format action-receipt/1

format · seal_suite	action-receipt/1 · elan1-seal-v6	all
sealed_fields · digest	F01–F16 and SHA-256 digest	all
signature · key_id	pure Ed25519; an identifier, never a key	CL-2
anchor	rfc3161-6962: path, root, leaves, token	CL-3
audience · nonce · expires_at	envelope, signed separately	CL-4
disclosure_suite · disclosures	elan1-sd-v1 openings (E23)	CL-D
issuer · limits	ar-org:<domain>; limits verbatim	all

Conformance, self-asserted with a published vector run:
CL-1 Structural · CL-2 Attested · CL-3 Anchored · CL-4 Addressed · CL-D
elan1 L1 → CL-1 only · L2 → CL-2 + CL-3 · L3 → CL-2 + 3 + 4 (+ CL-D)
Verifier: no network call, no code shared with a producer

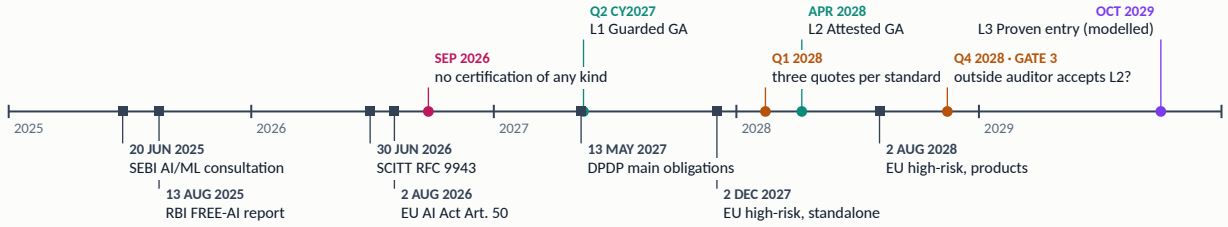
GOVERNANCE OF THE STANDARD — v0.x WITH elan1 AS EDITOR, THEN TRANSFER

Draft + change log — dated; every change names its comment
→ **60-day comment** — public; every comment answered in writing
→ **Two implementations** — built against the draft; results published
→ **Publish** — governed write, dual control; release key
→ **Transfer** — 3 implementations, ≥ 2 non-elan1 orgs → 12 mo

Refused: an unanswered comment, only our own implementation, or approval by the editor. Defects: acknowledged in 5 working days; erratum at once; a suite change is a new identifier; superseded, never withdrawn. Destination (IETF, W3C/OASIS, foundation) not chosen.

Landscape: SCITT is RFC 9943 (30 Jun 2026), COSE Receipts RFC 9942, SD-JWT RFC 9901; draft-marques-asqav-compliance-receipts-08 (31 Aug 2026). The IM recommends v1 as a SCITT profile on COSE with selective disclosure registered as a v1 suite (IM §9); E25 records no liaison work yet.

CERTIFICATION AND REGULATOR PATHWAY — elan1 PLAN ABOVE THE LINE (MODELLED), EXTERNAL DATES BELOW (PUBLISHED)



Certification rule (pre-committed)

Certify to the standard the first paying evidence customer's auditor names — never speculatively, never two in parallel:
India / EU, AI governance → ISO/IEC 42001 first (gaps: impact assessment, internal audit, management review)
US security review → SOC 2 Type I, then a Type II window
Budget: nil FY27 · ₹0.66 cr FY28 · ₹1.44 cr FY29 · ₹1.82 cr FY30 · ₹2.38 cr FY31 (modelled). A certificate changes no claim

Regulator engagement

Claim = three sentences, each mapped to an exit code, an artefact or a dated document; limits table handed over
Route in: consultations; a customer's supervisor; CA institutes; RBI, EU bodies only with a real regulated customer
Ask for corrections, report vocabulary, a view on the open format, a defect contact — never endorsement or a letter
Only the founder (with the elan1 lead) speaks to a regulator

published standard work · format (L3 era) · founder decision / gate · external party · ■ external date ● elan1 plan date

Source: E25 §01–§12, E26 §02–§04, E20 §02, IM §9, §10 · elan1 Technology Memorandum · Sept 2026

Figure 24 — The Action Receipt boundary: specification, reference verifier and vectors published under open licences while the governed write path, packs and profiles stay commercial; conformance levels CL-1 to CL-D mapped to assurance levels; and the separate certification (ISO/IEC 42001 or SOC 2) and regulator-engagement pathways, each bounded by what a verifier can demonstrate.

Explained in: 15. The Action Receipt standard, certification and regulator engagement (memorandum page 234) · Source file: fig-24-action-receipt-certification.svg

FIGURE 25 OF 32

Figure 25 — The assurance ladder and the upgrade path

L1 BEING BUILT · L2, L3 SPECIFIED, NOT BUILT

Same declaration at every level; what changes is rung 8 onward and what may be said. Dates are targets.

A level is a property of the proof and the claim, never of the policy: one declaration, three levels.
L1 → L2 by runbook (E19); L2 → L3 on an event.

L1 Guarded
BUILDING · Phase 1 · GA target Q2 CY2027

WHAT RUNS
- nine rungs; six gates never skippable
- evaluate() over versioned rules, no model
- autonomy A0–A5 (missing grant → A0)
- human approval; dual control at G4
- sandbox + 30-day rollback
- SHA-256 hash-chained record of every write, refusal and escalation
- 16-field layout frozen, unsigned
- meter: refusals at quantity 0 (DB CHECK)

MAY SAY
"checked against versioned rules before it happened; a person approved; tamper-evident to you, the chain holder"

MAY NOT SAY
signed · tamper-proof · immutable · independently verifiable · court-ready

BUYER Operations / finance owner: *"can I let this run?"*

GA GATE
E01 done-list 1–10 (1–7 hard gates: rung-skip suite, import-graph, nightly reconciliation, register lint, CHECK)
E09 readiness checklist rows 1–8 done

Plan list price ₹1,80,000 / tenant / yr

L2 Attested
FROZEN SPEC · Phase 3 · GA target Apr 2028

ADDS AT RUNG 8 AND AFTER
- Ed25519 over the 32-octet digest of the frozen preimage (absent, not stubbed, at L1)
- key custody per residency domain (KMS)
- Merkle root (RFC 6962) → RFC 3161 tokens from two TSAs, one India-based, ≤15 min
- eight-member bundle with manifest
- offline verifier: zero-dependency, free
- verifier profiles executing
- unanchored head shown on Assurance door

MAY SAY
"a party with our key fingerprint, got out of band, can verify this bundle on an air-gapped machine with no elan1 process"

MAY NOT SAY
accepted by MCA / RBI / SEBI · court-ready · immutable · verifies your books

BUYER The auditor's counterparty: *"can I hand this over?"*

DONE (E14, 12 measurements)
1–8 block: 64 vectors octet-identical in two impls; 28 negatives refused; key + successor; anchor SLO met for a month; two TSAs contracted; certified profile …

Plan list price ₹6,00,000 / tenant / yr

L3 Proven
FROZEN SPEC · Phase 4 · entry event-gated

SIX CAPABILITIES + THE STANDARD
- cross-organisation verification (E22)
- selective disclosure (E23)
- customer-held keys (E16, E23)
- sovereign deployment (deployment spec)
- precedent intelligence (E24)
- earned autonomy A2 → A3, revocable (E24)
- Action Receipt open standard (E25)
- second independent seal implementation
- non-elan1 counter-signature

MAY SAY
"the format is open; a receipt stays checkable after we are gone"

MAY NOT SAY
tamper-proof · regulator-approved · transaction genuine · "standardised"

BUYER The market: *"does this survive the vendor?"*

ENTRY CONDITION — AN EVENT
An outside auditor accepts an L2 bundle: ran the verifier · relied on it · says what PASS does not establish · would ask again
Planned entry Oct 2029 (modelled)

Plan list price ₹24,00,000 / tenant / yr

L1 → L2 UPGRADE RUNBOOK (E19) — NO RE-DECLARATION, NO MIGRATION, NO WRITE FREEZE

Seven preconditions, verified by replay and measurement, never asserted (E07 §06):
1 preimage unchanged since the first record (full-chain replay) · 2 key custody in the residency domain, with a registered successor · 3 anchor interval a published SLO, measured for a month · 4 offline verifier passes the vectors and refuses the tampered bundle · 5 certified profile for a regime the tenant is in · 6 whole-history chain continuity, scope and count printed · 7 proof_binding covers regime fields

0 · Pre-flight	1 · Key	2 · Signing on	3 · Anchoring on	4 · Profile	5 · First bundle	Close
replay whole chain: "continuous, 41,206 records"; record head seq	bind domain key, signing still off; fingerprint sent out of band first	governed write, dual_control: true; next record = seq B IRREVERSIBLE	first batch stamped by both TSAs; unanchored head falls to zero	block 6 addition only (new decl. version); certified profile; dry export	tenant exports and verifies offline: exit 0 PASS; tampered bundle exits 1	Flow 3: closing record in the chain

Boundary seq 1 … B−1 (e.g. 1–41,206): Level 1 forever, unsigned | seq B … (41,207 →): signature + key_id — prev_digest links across B; never back-signed

Rollback: stages 0, 1, 3, 4 fully reversible; after stage 2 signing can stop but signed records stay signed (a "signed island").

L1 (being built) · L2 (spec) · L3 (spec) · dual-control human step · tenant action · banned claims

Source: E07 §02–§06, E14 §02–§07, E19 §03–§06, E20 §02–§07, E01 §06, E09 §02, IM §1.3, §4.7, §8.1 · elan1 Technology Memorandum · Sept 2026

Figure 25 — The assurance ladder: what each level adds at rung 8 and after, the claim it permits, the entry gate to the next level, the five upgrade stages from L1 to L2, and the plan dates (all modelled).

Explained in: 16. Assurance levels and the upgrade path — L1 GA, L1→L2 and L2→L3 (memorandum page 249) · Source file: fig-25-assurance-ladder.svg

FIGURE 26 OF 32

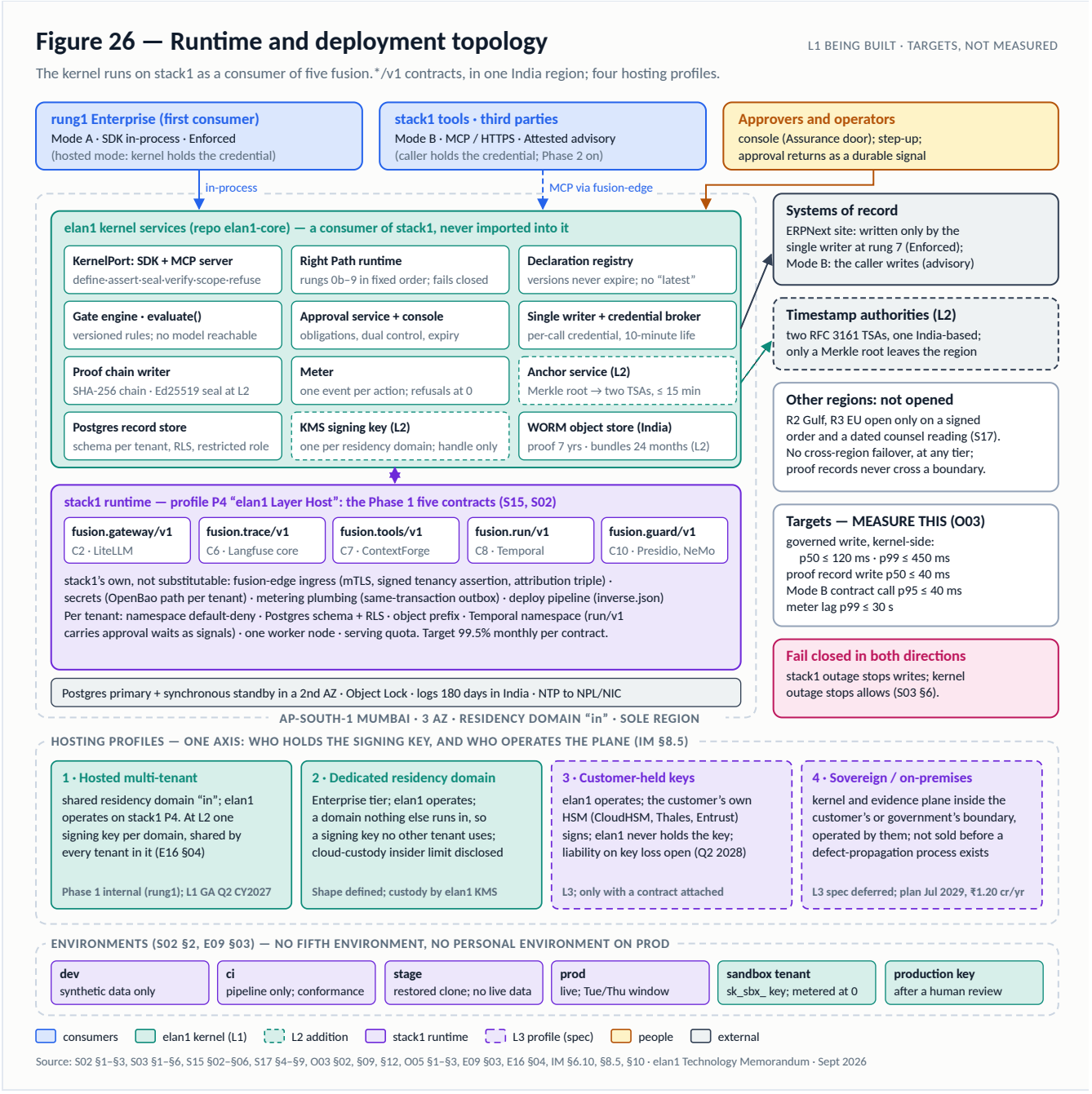


Figure 26 — Runtime and deployment: the kernel's units on stack1's P4 contract set inside the ap-south-1 residency boundary, the three planes and their absent routes, and the five hosting profiles from shared hosted to air-gapped, with who operates each and who holds the key.

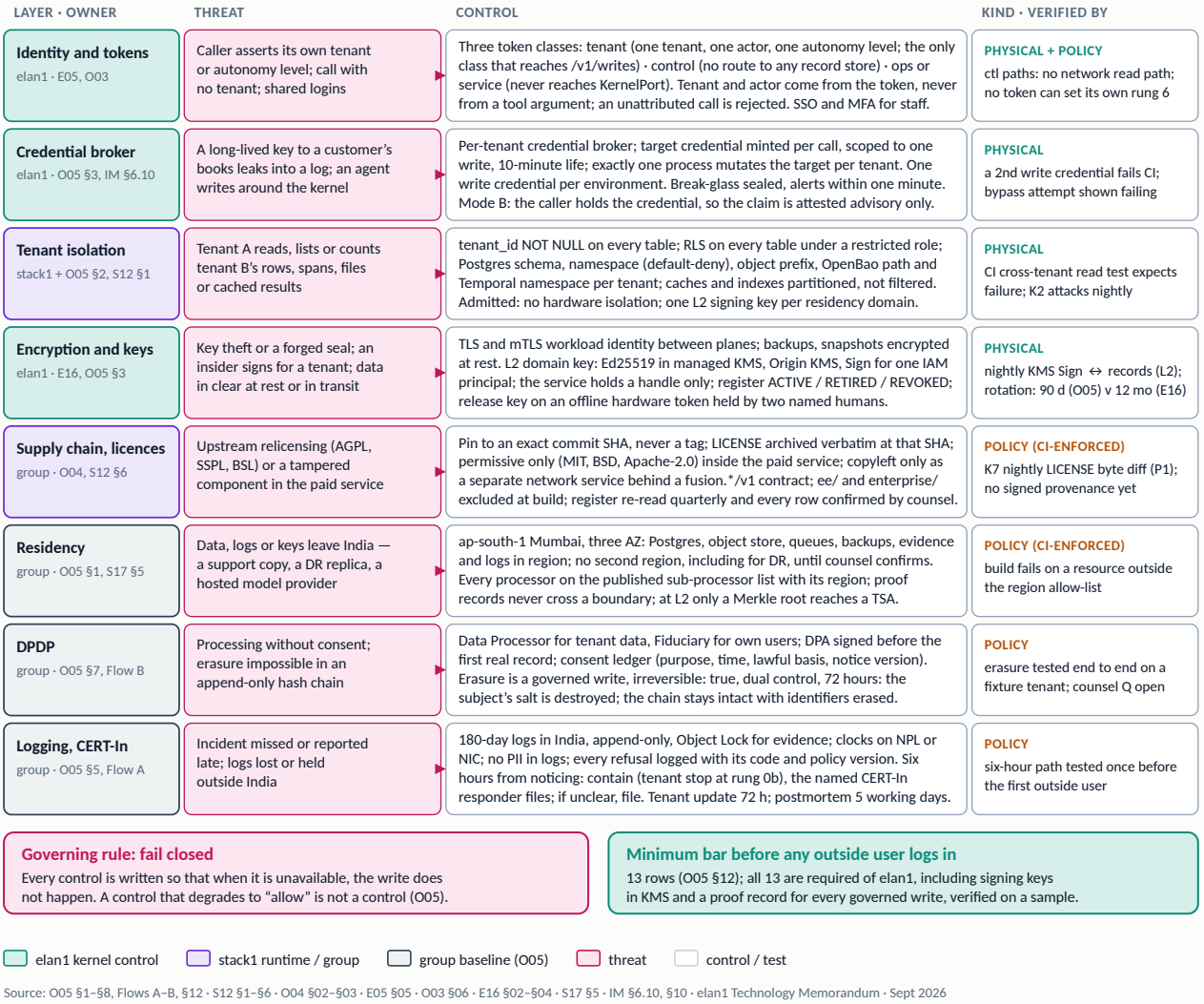
Explained in: 17. Runtime and deployment — stack1, hosting profiles and sovereign deployment (memorandum page 268) · Source file: fig-26-deployment-topology.svg

FIGURE 27 OF 32

Figure 27 — Security, residency and DPDP controls by layer

L1 BEING BUILT · DESIGNED, NOT AUDITED

Each threat mapped to its control and to the test that proves it; physical means the grant or route does not exist.



Source: O05 §1–§8, Flows A–B, §12 · S12 §1–§6 · O04 §02–§03 · E05 §05 · O03 §06 · E16 §02–§04 · S17 §5 · IM §6.10, §10 · elan1 Technology Memorandum · Sept 2026

Figure 27 — Security, residency and DPDP controls by layer: identity and token classes at the edge, the model-free decision path, the single writer and credential broker, the insert-only chain and salt store, KMS custody at L2, and the ap-south-1 residency boundary that contains all of them.

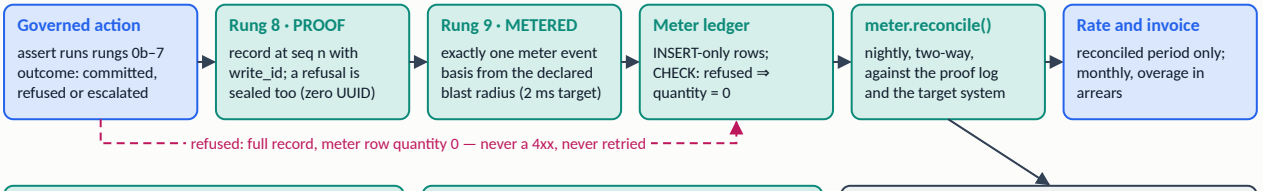
Explained in: 18. Security, residency, data protection and open-source licences (memorandum page 278) · Source file: fig-27-security-controls.svg

FIGURE 28 OF 32

Figure 28 — Metering pipeline: rung 9 to the invoice

DESIGNED — PRICES MODELLED, NOT QUOTED

One governed action is one record, one meter event and one invoice line; refusals are quantity 0 by database constraint.



Meter event (E02 §03)

event_id	uuid	kernel-assigned
write_id	uuid	always present
seq	uint64	the record it refers to
basis	enum	from blast radius
quantity	uint32	1 committed · 0 refused
outcome	enum	committed | refused | escalated
at	RFC 3339	platform clock

An invoice line with no write_id cannot exist: the schema has no such row.

Basis and the constraint

DECLARED BLAST RADIUS → METER BASIS (E10 §03)

one_record	→ per record
one_counterparty	→ per counterparty
batch	→ per batch
tenant	→ per tenant
cross_boundary	→ per crossing

Never configurable per customer; never discounted.

CHECK (outcome <> 'refused' OR quantity = 0)
SQL form is a build recommendation

Two-way reconciliation (E09 Flow 5, O07 §09)

every committed write has one record;
every record one write; every meter event references a write that exists; every refusal carries quantity 0
→ **any discrepancy stops the invoice, pages us**
kernel meter v credit ledger: zero tolerance, the kernel wins (P2 against the ledger)
runtime meter v component counters: 0.5%
unresolved at invoice time: bill the lower number, carry the gap as a named item

The invoice (illustrative layout; plan figures, modelled)

Tenant at L1 Guarded, India residency domain · one reconciled month

Platform floor	L1 ₹1,80,000 per tenant per year; not discountable
Governed actions in allowance	4,80,000 a year included at L1
Governed actions beyond it	× ₹1.60 (plan rate; L1 list price not yet set)
Refused 412	₹0 · count shown on the invoice, not a portal
Pack line (ledger1 …)	₹1,35,000 per tenant per year, blended
Evidence line (L2 onward)	verifier profile per regime per year; bundle export free within allowance; verifier ₹0

"Show the records behind this line" opens every event by write_id and seq.
A disputed line is held and settled against the proof record.

Related-party billing (IM §2.2, O07 §05)

elan1 → rung1: ₹0.40 per committed governed action from 1 Jan 2027; no floor; refusals ₹0; reset quarterly against measured cost. Own revenue line: engineering validation, not traction. Arm's-length basis documented.
stack1 → elan1: six runtime units at observed cost + 15% loading, booked as cost of revenue.

Three meters, one number each (O07 §03)

Kernel meter	elan1 · rung 9 · normative for every governed action
Runtime meter	stack1 · token, GPU, byte, page, tool units + attribution triple
Credit ledger	rung1 · a rendering, not a source; when it disagrees it is a defect

Rules that travel with the price (E10 §05, IM App. A.4)

Nobody may discount the platform floor, the meter basis or the price of a refusal; no discount on the L2 evidence line.
No per-seat price and no outcome price at any level: a refusal is a correct outcome with no business result.
The refusal rate is published beside cost per governed action every month, so a falling cost cannot hide a loosened gate.

consumer / billing · elan1 kernel, ledger, lines · L2 onward · reconciliation / meters · refusal and hard rules

Source: E02 §03–§04, E04 rung 9, E09 Flow 5, E10 §02–§06, O07 §02–§09, IM §2.2, §4.8, App. A.1–A.4 · elan1 Technology Memorandum · Sept 2026

Figure 28 — The metering pipeline: one meter event per governed action at rung 9, committed with the proof record; refusals, escalations and sandbox writes at quantity zero under a database CHECK; nightly two-way reconciliation against the hash-chained proof log; evidence and pack lines at rating; external invoices and the internal rung1 transfer statement produced from the same events.

Explained in: 19. Metering, pricing plumbing and billing (memorandum page 297) · Source file: fig-28-metering-pipeline.svg

FIGURE 29 OF 32

Figure 29 — Technology stack by layer

MOSTLY RECOMMENDATIONS · ADR BEFORE EACH WAVE

Chosen component, credible alternatives and what is rejected; where E/O documents override the product folder, they win.

LAYER	CHOSEN	ALTERNATIVES	REJECTED / AVOID	STATUS
Kernel language	Python 3.12 · FastAPI · uv (in repo); write API with generated OpenAPI 3.1	Go (static binary) Kotlin / Java 21 · TypeScript	—	RECOMMENDED revisit if p50 misses
Seal A and B	A: Python + pyca/cryptography B: TypeScript + node:crypto, a separate team	PyNaCl · Rust ed25519-dalek (verify_strict) · Java 21 EdDSA	one shared library for A and B	RULE FIXED E12; langs REC
Offline verifier	Go, standard library only (SHA-256, Ed25519, ASN.1/DER, X.509); empty lockfile	TypeScript on Node with no npm dependencies	Python, Rust, C/C++ (need dependencies)	ZERO-DEP FIXED E12, E15
Database	PostgreSQL 16+: role grants, FORCE RLS, CHECK; INSERT-only evidence tables	Cloud SQL / AlloyDB · Azure Postgres · self-managed Patroni	QLDB (retired) · any blockchain · Supabase prod	FIXED O05, E02
Outbox and queue	Postgres outbox, FOR UPDATE SKIP LOCKED, per-tenant commit order	Debezium CDC · LISTEN/NOTIFY as a wake-up hint	a broker: Kafka, Redpanda, NATS	LOAD-BEARING STS; ETS REC
Durable workflow	Temporal behind fusion.run/v1, a namespace per tenant; approvals arrive as durable signals	Prefect · Dagster (substitutes behind the same contract)	Windmill · n8n · Inngest (licences)	FIXED S04, DR-002
Policy engine	Own rules-as-data evaluator at rung 4: reads the projection and as_of only; no clock, no model	Cedar (fallback) · CEL · OPA/ Rego with time and http off	any model in the allow decision	RECOMMENDED ADR at W3
Key custody (L2)	AWS KMS, ECC_NIST_EDWARDS25519, pure Ed25519, Origin KMS, one key per residency domain	Google Cloud KMS (HSM level) · CloudHSM · Thales · Entrust (L3)	OpenBao / Vault transit (software keys)	NAMED E16 §03
Timestamp (L2)	eMudhra (India, CCA-licensed) plus one global TSA: DigiCert, Sectigo, GlobalSign or Entrust	the global partner is still to be chosen	FreeTSA and free TSAs (no SLA)	OPEN to 31 May 2027
MCP surface	Official MCP SDK in the API language; stateless 2026-07-28 revision; descriptions pinned by hash	hand-rolled JSON-RPC	—	RECOMMENDED E05, ETS
Console front end	React 18 · Vite · TypeScript · Tailwind; screens compiled from declarations, no screen files	Svelte / SvelteKit · Vue 3 · Radix primitives	hand-written screen files under app-*	IN REPO ETS §04
Secrets, identity	OpenBao path per tenant; Keycloak realm per tenant; mTLS workload identity (stack1)	AWS Secrets Manager · Doppler (ETS recommendation)	HashiCorp Vault (BUSL, blocked)	VIA STACK1 S03 overrides ETS
Observability	OpenTelemetry spans per rung → fusion.trace/v1; Grafana stack · Sentry · healthchecks.io	Datadog · Honeycomb · Opik / MLflow behind trace/v1	vendor agents (lock-in)	REC · PAGING vendor OPEN
CI/CD, supply chain	GitHub Actions + merge queue; Claude Code harness; cosign + SLSA; import-linter / depguard; CodeQL	GitLab CI · Buildkite · Snyk · Renovate or Dependabot	pinning by tag; ee/ and enterprise/ in builds	IN REPO O04, ETS §09
Hosting and IaC	stack1 managed Kubernetes in ap-south-1 only; Terraform or OpenTofu; Argo CD delivery	ECS Fargate (ETS) · Pulumi · GCP asia-south1	multi-cloud · a 2nd region before counsel	OPEN OD-28 vendor
Evidence store	Object Lock (WORM) in India: S3 Object Lock (ETS) or SeaweedFS / Ceph / Garage (STS)	GCS Bucket Lock · Azure immutable blob storage	MinIO (AGPL under the evidence plane)	OPEN ETS v STS

elan1 choice · inherited via stack1 contract · external vendor · rejected or avoided

Source: ETS §01–§13, STS §01–§02, S03 §1, S04 §01, O04 §03, O05 §1–§3, E05, E12, E15, E16 §03, E17, IM §6.10 · elan1 Technology Memorandum · Sept 2026

Figure 29 — The elan1 technology stack by layer: kernel-owned components (governed-write service, policy evaluator, chain writer, meter, registry, surfaces, and at L2 the seal, key register, anchor client and offline verifier) above the stack1 runtime substrate consumed through fusion.*/v1 contracts, with the chosen component, its principal alternative and the assurance level at which each first matters.

Explained in: 20. The technology stack — choices, alternatives and what was rejected (memorandum page 309) · Source file: fig-29-technology-stack.svg

FIGURE 30 OF 32

Figure 30 — Build roadmap, October 2026 to March 2031

PLAN — TARGETS, NOT MEASURED

Builds, gates, tranches and teams on one calendar. Every date is a plan target; revenue and tenant counts are modelled.

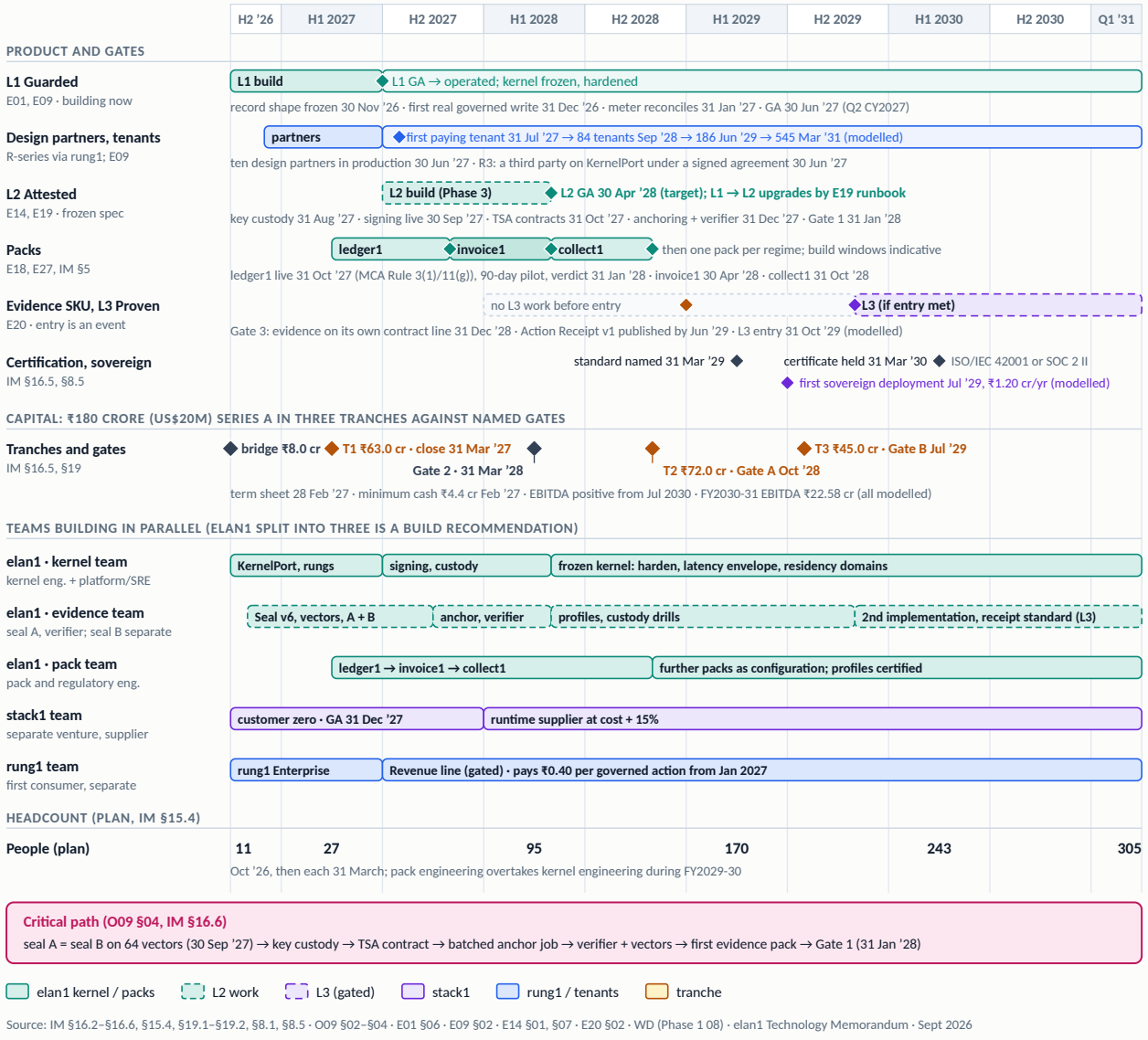


Figure 30 — Build roadmap, October 2026 to March 2031: the four phases, the elan1 kernel and evidence teams beside the stack1 and rung1 teams, the dated gates (first real governed write, L1 GA, L2 signing, anchoring, Gate 1, Gate 2, L2 GA, Gate 3, L3 entry) and the three tranches released against Gate A and Gate B; L1 is built now and L2 and L3 remain frozen specifications until their entry conditions.

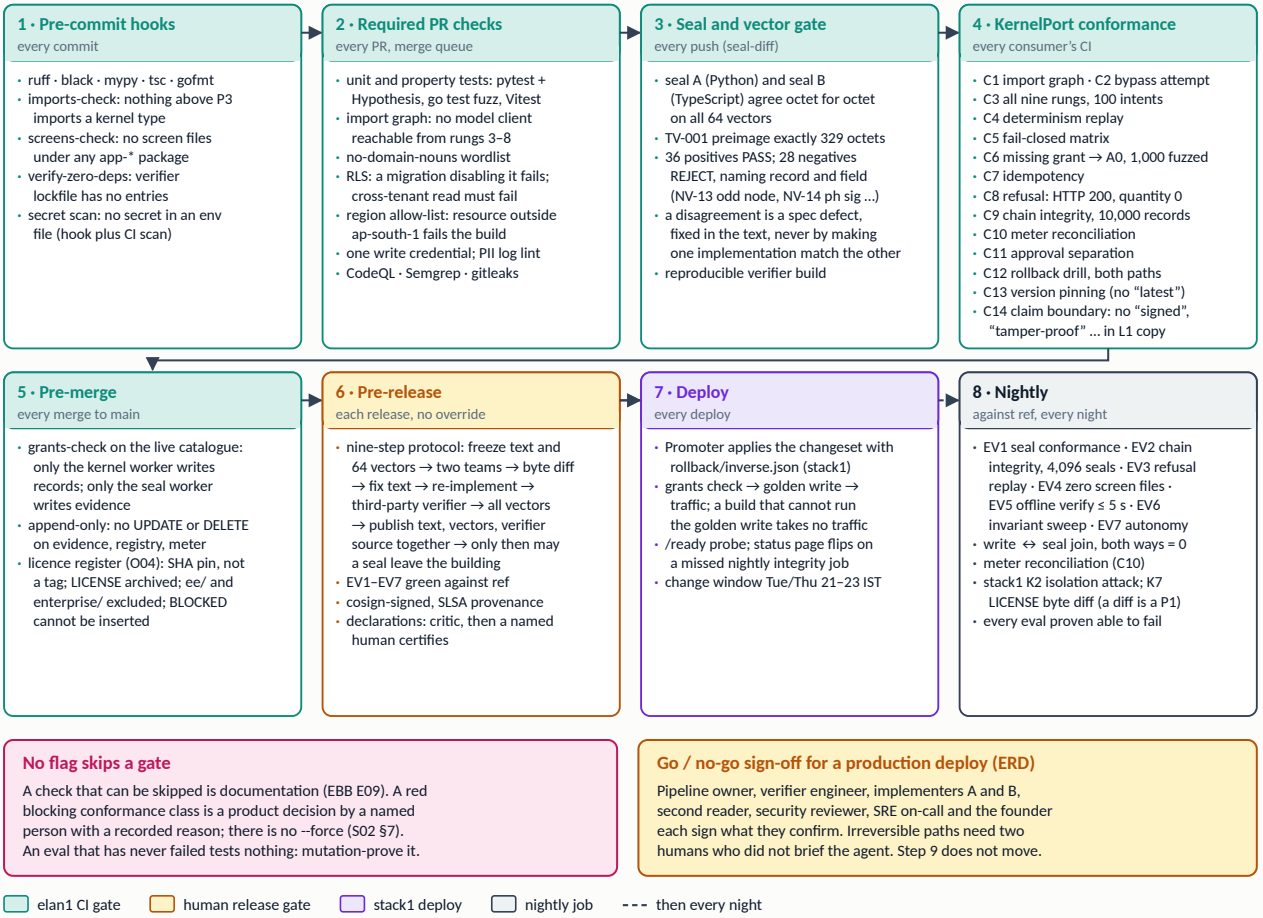
Explained in: 21. Engineering plan — teams, phases, build sequence and quality gates (memorandum page 323) · Source file: fig-30-build-roadmap.svg

FIGURE 31 OF 32

Figure 31 — CI pipeline and quality gates

DESIGNED — PIPELINE NOT YET RUNNING

From commit to traffic: eight gate stages, each a failing check rather than a review; none can be skipped with a flag.



Source: E02 §08, E12 §07–§08, EBB E09 and EB-05, ERD 6.11, 7.3, sign-off, O04 §01–§03, O05 §1–§2, S02 §7, S12 §6, IM §6.5 · elan1 Technology Memorandum · Sept 2026

Figure 31 — CI and quality gates: pre-commit (import graph and P3 boundary, lockfile, secrets), pull request (unit and property tests, register lint, schema lint, licence and ee/ path check, claim-boundary strings), push (digest vectors across two independent implementations), merge (grants check against stage, conformance C1–C14 on the reference tenant), nightly (replay, chain integrity, reconciliation, fail-closed matrix, evals), release (the seal release protocol from L2) and deploy (grants check, golden write, then traffic).

Explained in: 21. Engineering plan — teams, phases, build sequence and quality gates (memorandum page 332) · Source file: fig-31-ci-quality-gates.svg

FIGURE 32 OF 32

Figure 32 — Operations: SLOs, failure behaviour and incidents

DESIGNED — SLOs ARE TARGETS

Evidence integrity pages; latency does not. Every dependency failure refuses rather than allows; every incident has a runbook.

Nine SLOs (EBB E11, O03)

governed write p50	≤ 120 ms	MEASURE THIS
governed write p99	≤ 450 ms	MEASURE THIS
seal, rung 8, p50	≤ 40 ms	MEASURE THIS
outbox → seal lag p99	≤ 2 s	estimate
anchor interval (L2)	≤ 15 min	published SLO
bundle build, 10k seals	≤ 60 s	estimate
offline verify, 10k seals	≤ 5 s	estimate
writes without a seal	0	invariant
records mutated outside	0	invariant

Also: stack1 prod availability 99.5% a month per contract; meter lag p99 ≤ 30 s (S02 §3). None is quoted to a customer until measured under load.

Observability and paging

OpenTelemetry span per rung → fusion.trace/v1
status page per component: submission, gates, writer, chain, meter, console; a missed nightly integrity job flips chain itself

PAGES
outbox → seal lag over SLO · any non-zero invariant · TSA down two intervals · platform-class refusals above baseline · an eval failure

TICKETS
write latency · bundle build · Studio · connectors. One rotation, two people.

Kernel unreachable: fail closed

consumer: no degraded mode, no cached allow, no local gate (R12 RB-4, O03)
runs parked durably with the same idempotency key; SEAL_PENDING first
user: "We cannot check this change right now. Nothing was changed."

INSIDE THE KERNEL (IM §6.10, E04, E16, E17)
registry down → refuse at write door
policy down → refuse at rung 4, page
target down → refuse after sandbox
chain append fails → outbox retries, no gap
KMS down (L2) → domain refuses, no stale key
TSA down (L2) → writes go on, single_anchor

INCIDENT CLASS	DETECTED BY	RUNBOOK AND FIRST ACTIONS	SEVERITY
Proof-chain gap	nightly write ↔ seal join; CHAIN_HEAD_CONFLICT	RB3: never backfill a seal; write a sealed gap record stating the write_id, time and reason. Two chain writers stop the tenant.	P0
Seal worker behind	outbox → seal lag p99 over 2 s	RB1: confirm the lag, then decide whether to stop intake; stopping intake is the default, because a gap does not recover.	page
Meter / proof mismatch	nightly two-way reconciliation (C10)	The invoice for the period is stopped and we are paged; the kernel count wins over the credit ledger; a disputed line is held.	P2 → page
Key compromise (L2)	unaccounted KMS Sign call; exposed credential	RB6 / E16 Flow 2: fix T_suspect; stop signing, keep writing at L1; revoke; tenants told in writing within 24 h; anchors as witness.	P0
TSA outage (L2)	token timeout or malformed token	RB2 / E17 Flow 2: single_anchor; page at 2 intervals; banner at 4; late batches anchored oldest first, never merged. No write refused.	degraded
Declaration mis-publish	certifier or tenant report	RB10: a declaration version must be revoked; RB7 stops a tenant. Loosening a gate needs a new version plus an ADR (IM §6.11).	not stated
Isolation breach	K2 nightly attack; RLS test; customer report	O05 Flow A: tenant stop at rung 0b, proved by a refused attempt; revoke credentials; CERT-In responder assesses; DPDP check.	P0
Bundle fails at customer	customer or auditor report (L2)	RB9: reproduce air-gapped with our signed verifier first; outdated verifier, inconsistent bundle, or a spec defect disclosed to all.	P0 if real

Severity, CERT-In and notification

P0: compromise, data exposure, tamper evidence or fleet outage: response 15 min 24×7, contained in 2 h (R12)
CERT-In: the clock starts on noticing; the named responder files within 6 h; if reportability is unclear, file
Tenant update within 72 h; written note for every P0 and P1 to affected tenants within 5 working days (E09)

Backup, restore and DR (O05 §6, S09 RB-07, S17 §9)

Point-in-time recovery; nightly encrypted backups kept 30 days
Quarterly restore drill into a clean environment, timed; it restores the evidence store and re-verifies a sample
No second region, so no cross-region failover: a region loss is a restore in place from in-region backups
RPO and RTO not yet set (open, due 15 Oct 2026)

kernel / evidence · runtime / isolation · external dependency · customer-raised · P0 / fail-closed · people and process

Source: EBB E11 · S09 RB-01–RB-09 · R12 §3, RB-4 · E04 rungs 7–8 · E09 §02 · E16 Flow 2 · E17 Flow 2 · O05 §6, Flow A · O03 §03 · IM §6.10 · elan1 Technology Memorandum · Sept 2026

Figure 32 — Operations for the elan1 kernel: the service levels and invariants on the operations surface, the nightly integrity and reconciliation jobs, what pages and what raises a ticket, the incident flow from notice to verified close, fail-closed behaviour when the kernel or a dependency is unreachable (Mode A stops, Mode B relies on the caller), and single-region backup with quarterly restore drills that re-verify the proof chain.

Explained in: 22. Operations, reliability and incident response (memorandum page 339) · Source file: fig-32-operations-incident.svg